
Helios

**Lumiere Cinema
Software Architecture Document**

Version 1.5

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

Revision History

Date	Version	Description	Author
18/06/2025	1.0	Write Introduction	Ngô Hồng Thanh
		Write Goals and Constraints	Vương Ngũ Tín Thành
		Write Logical View (overview)	Nguyễn Thiên Nhã Trân
19/06/2025	1.1	Write Administrator service	Phan Nhựt Anh
		Write Authentication service , User service , Movie service , Branch service , Ticket service , Report service	Ngô Hồng Thanh
		Write Component: Hooks	Phan Nhựt Anh
20/06/2025	1.2	Write UI Overview	Phan Nhựt Anh
		Write Movie navigation , User profile , Dashboard layout	Ngô Hồng Thanh
		Write Ticket booking flow , Authentication layout	Nguyễn Thiên Nhã Trân
23/06/2025	1.3	Write Route Overview	Tạ Thiên Lâm
		Write Authentication route	Ngô Hồng Thanh
		Write User route , Move route , Branch route , Report route , Administration route	Tạ Thiên Lâm
		Write Ticket route	Vương Ngũ Tín Thành
		Write Controller Overview	Vương Ngũ Tín Thành
		Write Authentication controller , User controller	Ngô Hồng Thanh
		Write Movie controller , Branch controller , Branch controller , Report controller , Chatbot controller , Administrator controller .	Tạ Thiên Lâm
		Write Ticket controller	Nguyễn Thiên Nhã Trân
26/06/2025	1.4	Write Component: Models	Vương Ngũ Tín Thành
		Write Component: Database	Vương Ngũ Tín Thành
		Write Component: Large Language Model	Tạ Thiên Lâm
28/06/2025	1.5	Draw Deployment Graph	Nguyễn Thiên Nhã Trân
		Write Client-side & Server-side	Vương Ngũ Tín Thành
		Write Third-Party Services Integration	Ngô Hồng Thanh
		Write Implementation view: Back-end	Tạ Thiên Lâm
		Write Implementation view: Front-end	Phan Nhựt Anh

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

Table of Contents

- 1. Introduction..... 4**
 - 1.1 Purpose..... 4
 - 1.2 Scope..... 4
 - 1.3 Overview..... 4
- 2. Architectural Goals and Constraints..... 4**
 - 2.1 Architectural goals..... 4
 - 2.2 Architectural constraints..... 5
- 3. Use-Case Model..... 6**
- 4. Logical View..... 7**
 - 4.1 Component: Services..... 7
 - 4.2 Component: Hooks..... 11
 - 4.3 Component: User Interface (UI)..... 15
 - 4.4 Component: Routers..... 20
 - 4.5 Component: Controllers..... 26
 - 4.6 Component: Model..... 30
 - 4.7 Component: Database..... 36
 - 4.8 Component: Large Language Model..... 37
- 5. Deployment..... 38**
 - 5.1 Client-side..... 38
 - 5.2 Server-side..... 38
 - 5.3 Third-Party Services Integration - Large Language Model (LLM)..... 38
- 6. Implementation View..... 39**
 - 6.1 Front-end..... 39
 - 6.2 Back-end..... 40

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

Software Architecture Document

1. Introduction

The Software Architecture Document provides a comprehensive overview of the system's architectural design. It serves as a blueprint for the development team, stakeholders, and future maintainers, offering insights into the high-level structure, major components, their interactions, and key architectural decisions.

1.1 Purpose

The purpose of this document is to describe the software architecture of the Lumiere Cinema system. It aims to ensure that developers, testers, designers, and project managers, have a shared understanding of the system architecture. This document will support decision-making during development, guide implementation, and facilitate communication among team members.

1.2 Scope

This Software Architecture Document outlines the overall architecture of the Lumiere Cinema system, focusing on its major components, modules, interfaces, and the way they interact. It explains the structural composition of the system, the core design principles, and the rationale behind the selection of architectural styles and patterns.

1.3 Overview

The following sections provide a comprehensive breakdown of the architecture of the Lumiere Cinema system. The document is organized as follows:

- **Introduction:** Provides an introduction to the Software Architecture Document, its purpose, scope and an overview of the content.
- **Architectural Goals and Constraints:** Identifies the key goals and limitations that shaped architectural decisions.
- **Use-Case Model:** Presents the architectural perspective from the standpoint of system use cases and scenarios.
- **Logical View:** Details the logical organization of the system, emphasizing the division of responsibilities among different components.
- **Deployment:** Demonstrates how software components are deployed onto the physical hardware environment.
- **Implementation View:** Describes how the components are implemented and organized in the codebase.

2. Architectural Goals and Constraints

This section describes the primary goals and constraints that significantly impact the architectural design of the Lumiere Cinema system. These are derived from the non-functional requirements and the project's development environment.

2.1 Architectural goals

Architectural goals are the critical non-functional requirements that the system must achieve, having a major influence on how the system is designed.

- High performance:
 - + Ensure that the system responds to user actions (e.g., login, movie search, seat selection, payment) within 3 seconds for 95% of cases.
 - + Maintain system stability and responsiveness even during peak traffic periods, especially during new movie releases or prime booking hours.
- Scalability:
 - + The architecture must be easily scalable to accommodate future increases in the number of users, cinema branches, and transaction volumes without significant structural changes.
 - + Support the addition of new features without substantially impacting existing functionalities.
- Reliability & Availability:
 - + The system must exhibit good fault tolerance, providing clear error messages and recovering gracefully in case of failures.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

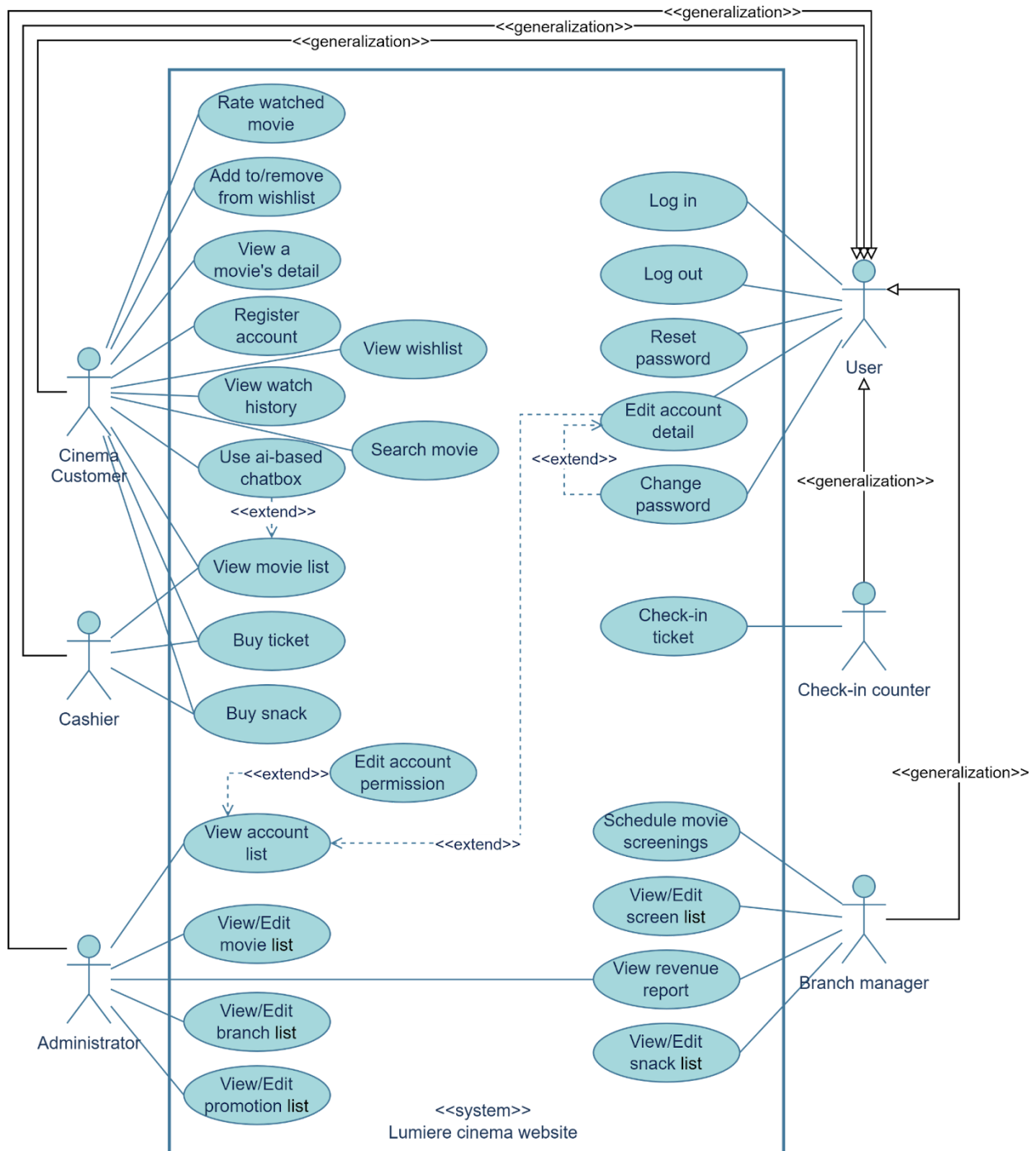
- + Guarantee data integrity and security, allowing changes only through authorized channels.
- Security:
 - + Implement robust role-based access control (RBAC) for users, staff, and administrators.
 - + Log all significant user interactions and system activities for auditing purposes and anomaly detection.
- Usability:
 - + Ensure the user interface is responsive and optimized for various devices and screen sizes (desktop, laptop, tablet, smartphone).
 - + Support popular modern web browsers (Chrome, Firefox, Edge).
 - + Provide a clear, intuitive, and easy-to-use interface for all user types.
- Maintainability:
 - + The architecture should be modular, easy to understand, modify, and upgrade.
 - + The codebase must adhere to good design principles and be well-documented.

2.2 Architectural constraints

- Language and technology stack:
 - + The website is built by using HTML, CSS, JavaScript and NoSQL.
 - + This project uses ReactJS and TailwindCSS for front-end; Express and NodeJS for back-end; MongoDB and Mongoose for Database management.
- Deployment environment: The primary application will be a web application, requiring compatibility with major web browsers.
- Development tools: Jira for project management, GitHub for source code version control, Google Drive for documentation storage, and Figma for software prototyping.

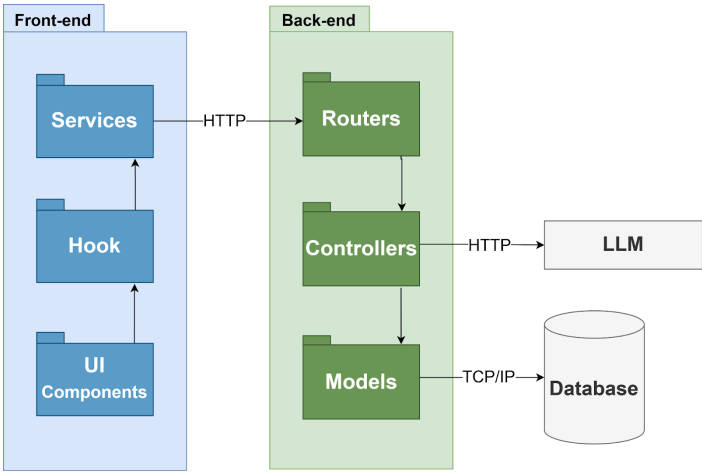
Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

3. Use-Case Model



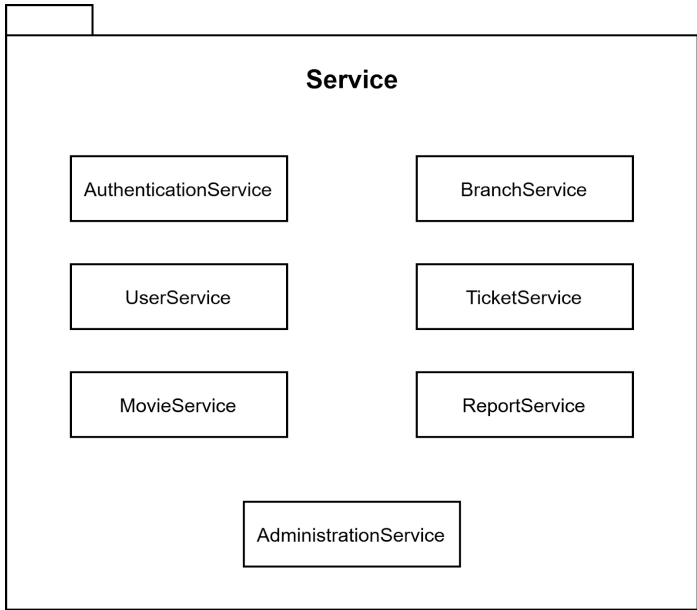
Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4. Logical View



The architecture of this software references two popular models, corresponding with the programming framework. The front-end is structured based on the three-layer architecture in ReactJS, including services, hooks and UI components. The back-end is structured based on the MVC architecture, including routers, controllers and models. The front-end and back-end communicate with each other through the front-end's service and back-end's routers.

4.1 Component: Services



In the front-end architecture of the Lumiere Cinema, services are dedicated modules that encapsulate communication with back-end APIs. Each service is responsible for sending HTTP requests (GET, POST, PUT/PATCH, DELETE) and receiving responses, without handling any UI state or rendering logic. Services act as a separation layer between the user interface (components, hooks) and the back-end. This abstraction ensures that:

- API logic is centralized, reusable, and easy to test.
- Hooks can focus on managing UI state, side effects, and error handling.
- Changes to API endpoints or formats only affect the services, not the entire application.

Each service corresponds to a specific domain of functionality and exposes functions that are called internally by custom React hooks, such as `useAuthentication`, `useUser`, `useTicket`,...

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.1.1 Authentication service

AuthenticationService
+ login() + register() + resetPassword() + changePassword() + logout()

Authentication Service handles communication with the back-end for user authentication features.

Functions:

login(): Sends a POST request to the back-end with login credentials and receives an auth token if valid.

register(): Sends a POST request to register a new user account with name, email, and password.

resetPassword(): Sends a POST request for a password reset, which triggers an email with reset instructions.

changePassword(): Sends a POST request to update the password after verifying the current one on the server.

logout(): Logs the user out of the system.

4.1.2 User service

UserService
+ getProfile() + updateProfile() + getWatchHistory() + getWishlist() + addToWishlist() + removeFromWishlist() + rateMovie() + getRatings()

User Service provides methods for interacting with back-end APIs related to user profile, watch history, wishlist, and movie ratings.

Functions:

getProfile(): Sends a GET request to the back-end to retrieve the current user's profile information.

updateProfile(): Sends a PUT/PATCH request to update profile fields (e.g. name, avatar) on the server.

getWatchHistory(): Requests watch history data from the back-end tied to the current user.

getWishlist(): Fetches the current user's wishlist by sending a GET request to the server.

addToWishlist(): Sends a POST request to add a movie to the user's wishlist on the server.

removeFromWishlist(): Sends a DELETE request to remove a movie from the wishlist in the back-end.

rateMovie(): Sends a POST request to submit a rating for a movie, linked to the current user.

getRatings(): Sends a GET request to retrieve the user's rating(s). May be filtered by movie.

messageChatBot(): Sends a POST request to retrieve the LLM model's response based on the user's messages/request.

4.1.3 Movie service

MovieService
+ getNowShowing() + getUpcoming() + getMovieDetail() + getShowtimes() + searchMovies() + getMovieRatings() + getAllMovies() + addMovie() + updateMovie() + deleteMovie()

Movie Service exposes functions to fetch and filter movie-related data from the back-end, including listings, search results, and showtimes.

Functions:

getNowShowing(): Sends a GET request to retrieve currently playing movies from the server.

getUpcoming(): Fetches a list of upcoming movies by querying the back-end.

getMovieDetail(): Sends a GET request to retrieve detailed movie data by ID.

getShowtimes(): Fetches showtimes from the back-end for a given movie.

searchMovies(): Sends a GET request to the back-end to find movies by keyword.

getMovieRatings(): Sends a GET request to retrieve the rating for a movie.

getAllMovies(): Sends a GET request to retrieve all movies (including inactive ones) for admin management.

addMovie(): Sends a POST request to add a new movie.

updateMovie(): Sends a PUT/PATCH request to update details of an existing movie.

deleteMovie(): Sends a DELETE request to remove or deactivate a movie from the system.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.1.4 Branch service

BranchService
+ getBranches() + getScreens() + getScreenDetail() + getSchedules() + getScheduleDetail() + getSnacks() + getSnackDetail() + scheduleMovie() + addBranch() + addScreen() + addSnack() + updateBranch() + updateScreen() + updateSnack() + updateSeatLayout() + updateSchedule() + removeBranch() + removeScreen() + removeSnack() + removeSchedule()

Branch Service manages back-end interaction for branch-specific features such as screens, schedules, snack menus, movies, and staff.

Functions:

getBranches(): Sends a GET request to retrieve all branches in the cinema system.

getScreens(): Fetches a list of screens from the specified branch via API.

getScreenDetail(): Sends a GET request to retrieve detailed information for a specific screen within a branch for management purposes

getSchedules(): Sends a request to get movie showtimes at the branch for a given date.

getScheduleDetail(): Sends a GET request to retrieve detailed information for a specific movie schedule within a branch for management purposes.

getSnacks(): Retrieves available snacks and combos at a given branch from the back-end.

getSnackDetail(): Sends a GET request to retrieve detailed information for a specific snack item within a branch for management purposes.

scheduleMovie(): Sends a POST request to create a schedule/showtime for a movie on a given screen.

addScreen(): Sends a POST request to create a new cinema screen within a specific branch.

addSnack(): Sends a POST request to add a new snack item to a specific branch's snack list.

updateScreen(): Sends a PUT/PATCH request to update details of an existing screen within a specific branch.

updateSnack(): Sends a PUT/PATCH request to update details of an existing snack item for a specific branch.

updateSeatLayout(): Sends a PATCH request to update the entire seat layout and pricing configuration for a specific screen.

updateSchedule(): Sends a PUT/PATCH request to update the movie screening at a specific screen and time for a branch.

removeScreen(): Sends a DELETE request to remove a specific screen from a specific branch.

removeSnack(): Sends a DELETE request to remove a snack item from a specific branch's list.

removeSchedule(): Sends a DELETE request to cancel an existing movie schedule for a specific branch.

4.1.5 Ticket service

TicketService
+ getAvailableSeats() + applyPromotion() + startHoldSession() + clearSession() + createMovieTicket() + getMovieTicketDetailAdmin() + updateMovieTicket() + deleteMovieTicket() + createSnackTicket() + getSnackTicketDetailAdmin() + updateSnackTicket() + deleteSnackTicket() + checkin() + reactivateTicket()

Ticket Service handles back-end communication for the ticket booking process including seat availability, promotions, and ticket creation.

Functions:

getAvailableSeats(): Sends a GET request to fetch seat availability for a specific movie schedule.

applyPromotion(): Sends a POST request to validate a promo code and returns the applicable discount.

startHoldSession(): Sends a POST request to temporarily reserve seats for the duration of the booking session.

clearSession(): Sends a DELETE or cancel request to release held seats on timeout or user cancellation.

createMovieTicket(): Sends a POST request to book seats for a movie schedule by customer or cashier.

getMovieTicketDetailAdmin(): Fetches detailed data of any movie ticket by ID for admin or staff.

updateMovieTicket(): Sends a PATCH request to update ticket info, such as status, by authorized personnel.

deleteMovieTicket(): Sends a DELETE request to cancel or remove a movie ticket logically (admin use).

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

createSnackTicket(): Sends a POST request to purchase snack items by customer or cashier.
getSnackTicketDetailAdmin(): Fetches details of any snack ticket by ID for staff or admin view.
updateSnackTicket(): Sends a PATCH request to update status or details of a snack ticket (admin use).
deleteSnackTicket(): Sends a DELETE request to cancel or logically delete a snack ticket (admin use).
checkin(): Sends a PATCH request to mark a ticket as checked in at the cinema.
reactivateTicket(): Sends a PATCH request to reactivate a ticket for use.

4.1.6 Report service

ReportService
+ getTickets() + getAvailableBranches() + getAvailableMovies()

Report Service handles communication with back-end reporting endpoints. Provides revenue data, supports export operations, and fetches available filtering options (branches, movies).

Functions:

getTickets(): Sends a GET request to retrieve all the tickets.

getAvailableBranches(): Sends a GET request to retrieve branch options available for reporting filters.
getAvailableMovies(): Sends a GET request to retrieve a list of movies for the selected branch.

4.1.7 Administration service

AdministrationService
+ getAllUsers() + getUserDetail() + addUser() + updateUserProfile() + deleteUser() + updateUserPermissions() + getAllPromotions() + addPromotion() + updatePromotion() + deletePromotion() + addBranch() + updateBranch() + removeBranch() + addMovie() + updateMovie() + removeMovie()

AdministrationService is responsible for handling back-end API communication related to system-wide administrative operations. This includes user account management, schedules, branches, staff, movies, snacks, and promotions. This service is exclusively used by administrator-level hooks like useAdministrator.

Function:

getAllUsers(): Sends a GET request to retrieve all user accounts in the system for admin management.

getUserDetail(): Fetches detailed profile data of a specific user account by ID.

addUser(): Sends a POST request to add a new user account.

updateUserProfile(): Sends a PATCH request to update the profile information of a user.

deleteUser(): Sends a DELETE request to remove a user account from the system.

updateUserPermissions(): Sends a POST request to assign or remove roles from a specific user account.

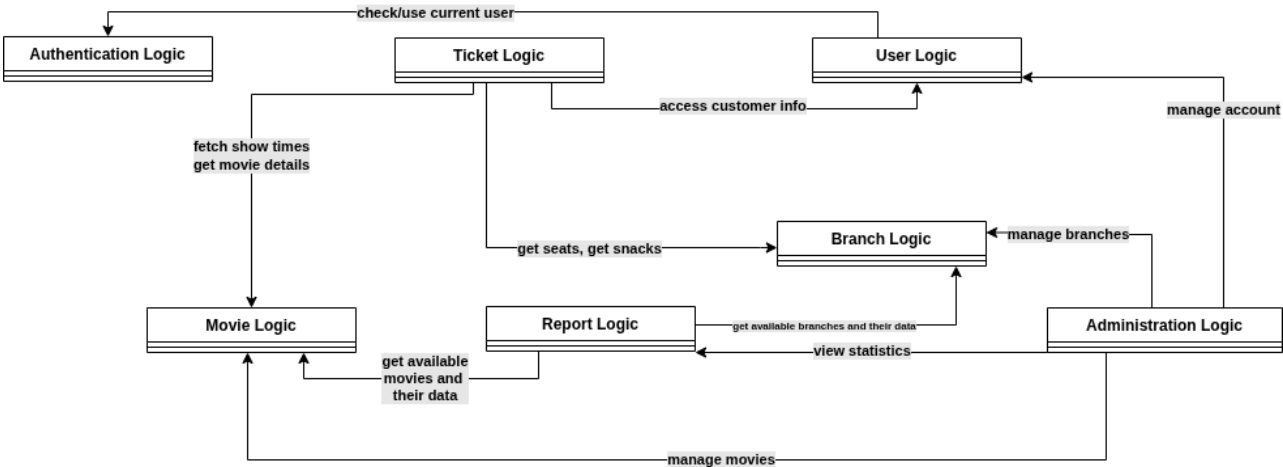
getAllPromotions(): Retrieves all promotional campaigns, including expired or inactive ones, for admin review.

addPromotion(): Sends a POST request to create a new promotional campaign in the system.
updatePromotion(): Sends a PUT/PATCH request to fully update all details of a specific promotion.
deletePromotion(): Sends a DELETE request to remove or deactivate a promotional campaign.
addBranch(): Sends a POST request to add a new cinema branch to the system.
updateBranch(): Sends a PUT/PATCH request to update all high-level details of an existing cinema branch.
removeBranch(): Sends a DELETE request to remove a cinema branch from the system.
addMovie(): Sends a POST request to add a new movie.
updateMovie(): Sends a PUT/PATCH request to update details of an existing movie.
removeMovie(): Sends a DELETE request to remove or deactivate a movie from the system.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.2 Component: Hooks

The Hook layer in the front-end serves as an intermediary between the user interface (UI) and service layers. Each custom hook is organized by domain (e.g., movies, tickets, users, administration) and is responsible for managing local state, calling APIs, and handling business logic on the client side. Hooks are designed with single-responsibility in mind – each function performs a specific task and avoids triggering UI side effects such as navigation or notifications. This separation ensures hooks are easy to reuse across components, simple to unit test, and flexible to extend as the system evolves. Additionally, user-facing and admin-facing hooks are clearly separated to maintain code clarity, enforce permission boundaries, and support scalable front-end architecture.



4.2.1 Authentication logic

AuthenticationLogic
- currentUser - authToken - error - loading
+ useLogin() + useRegister() + useLogout() + useResetPassword() + useChangePassword()

These hooks handle all user authentication logic on the front-end, including: login, registration, logout, password reset, token-based user info retrieval, and login status checking.

Functions:

- useLogin():** Sends login credentials to the server. If valid, stores the token and user info into state.
- useRegister():** Sends user registration data (name, email, password...) to the server to create a new account.
- useLogout():** Logs out the user: clears token, resets state, and optionally clears localStorage.

- useResetPassword():** Sends a password reset request to the user’s email. The server will send a reset link.
- useChangePassword():** Changes password for the logged-in user, using the current password for verification.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.2.2 User logic

UserLogic
- userProfile - role - watchHistory - wishlist - loading - error
+ useFetchProfile() + useUpdateProfile() + useGetWatchHistory() + useGetWishlist() + useAddToWishlist() + useRemoveFromWishlist() + useRateMovie() + useGetMyRatings() + useMessageChatBot()

These hooks manage user profile data and user-specific interactions such as watch history, wishlist, and movie ratings.

Functions:

useFetchProfile(): Retrieves the current user's profile from the server.

useUpdateProfile(): Sends updated profile data to the server.

useGetWatchHistory(): Retrieves the user's past movie viewing history.

useGetWishlist(): Retrieves the list of movies the user has added to their wishlist.

useAddToWishlist(): Adds a movie to the user's wishlist.

useRemoveFromWishlist(): Removes a movie from the wishlist.

useRateMovie(): Submits a star rating for a specific movie.

useGetMyRatings(): Retrieves the rating the user has given to a specific movie.

useMessageChatBot(): Sends a message to the AI assistant and returns the generated response for the user.

4.2.3 Movie logic

MovieLogic
- currentBranch - movieList - filteredMovies - selectedMovie - showtimes - loading - error
+ useFetchBranches() + useFetchNowShowing() + useFetchComingSoon() + useGetMovieDetail() + useGetShowtimes() + useSearchMovies() + useFilterByBranch()

These hooks handle movie-related logic such as retrieving movie lists, searching, filtering, and getting showtimes based on selected branches.

Functions:

useSetBranch(): Sets the current working branch to fetch data accordingly.

useFetchNowShowing(): Fetches the list of movies currently showing.

useFetchComingSoon(): Fetches the list of upcoming movies.

useGetMovieDetail(): Retrieves detailed information about a specific movie.

useGetShowtimes(): Fetches the showtimes for a specific movie at the current branch.

useSearchMovies(): Searches for movies matching the keyword.

useFilterByBranch(): Filters the movie list based on a specific branch.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.2.4 Branch logic

BranchLogic
- branchList - currentBranch - screens - schedules - snacks - loading - error
+ useFetchBranches() + useSetCurrentBranch() + useGetScreens() + useUpdateScreen() + useRemoveScreen() + useGetSchedules() + useUpdateSchedule() + useRemoveSchedule() + useGetSnacks() + useUpdateSnack() + useRemoveSnack()

These hooks handle all branch-level data operations such as fetching screens, schedules, snacks, and managing movie/staff within a specific branch.

Functions:

useFetchBranches(): Retrieves the list of branches the user can access.

useSetCurrentBranch(): Sets the currently selected branch for contextual operations.

useGetScreens(): Retrieves the list of screens for the currently selected branch.

useUpdateScreen(): Updates screen details such as name, type, or capacity.

useRemoveScreen(): Deletes a screen from the selected branch.

useGetSchedules(): Retrieves scheduled movies for the current branch on a given date.

useUpdateSchedule(): Updates or creates a showtime schedule for a movie in the branch.

useRemoveSchedule(): Removes a scheduled showtime from the branch.

useGetSnacks(): Fetches the list of snacks available at the selected branch.

useUpdateSnack(): Updates snack information such as name, price, or availability.

useRemoveSnack(): Deletes a snack item from the branch's menu.

4.2.5 Ticket logic

TicketLogic
- selectedSeats - selectedSnacks - schedule - appliedPromotion - customer - totalPrice - holdTimer
+ useFetchAvailableSeats() + useSelectSeat() + useRemoveSeat() + useSelectSnack() + useCalculateTotal() + useApplyPromotion() + useCreateTicket() + useStartHoldSession() + useClearSession() + checkin() + activeTicket()

These hooks manage the entire ticket booking logic on the front-end, including seat selection, snack selection, promotions, and ticket creation.

Functions:

useFetchAvailableSeats(): Retrieves the available seats for the selected schedule.

useSelectSeat(): Adds a seat to the selected seats list.

useRemoveSeat(): Removes a seat from the selected seats list.

useSelectSnack(): Adds or updates the selected snack quantity.

useCalculateTotal(): Calculates the total price including tickets, snacks, and promotions.

useApplyPromotion(): Applies a promotion code to the current booking.

useCreateTicket(): Sends the booking information to the server to create a ticket.

useStartHoldSession(): Initiates a session to temporarily hold selected seats.

useClearSession(): Clears all selected seats and snacks.

checkin(): Marks a ticket as checked in at the cinema.

activeTicket(): Activates a reserved or paid ticket for use.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.2.6 Report logic

ReportLogic
- isAdmin - reportData - chartData - exportStatus - selectedBranch - selectedRange - selectedMovie - availableBranches - availableMovies - loading - error
+ useGetRevenueReport() + useExportReport() + useGetAvailableBranches() + useGetAvailableMovies() + useSetBranch() + useSetDateRange() + useSetMovie()

These hooks handle fetching and filtering of revenue and performance reports by branch, movie, or date, with role-based access control.

Functions:

useGetRevenueReport(): Retrieves report data based on selected filters.

useExportReport(): Exports the report to a specified format.

useGetAvailableBranches(): Returns the list of branches available for reporting.

useGetAvailableMovies(): Returns list of movies for the selected branch.

useSetBranch(): Sets the reporting context to a selected branch.

useSetDateRange(): Sets the date range for report filtering.

useSetMovie(): Sets the selected movie for filtering report results.

4.2.7 Administration logic

AdministrationLogic
- accountList - selectedUser - allRoles - promotionList - movieList - loading - error
+ useGetAccounts() + useAddAccount() + useUpdateAccount() + useRemoveAccount() + useUpdateUserPermission() + useGetPromotions() + useAddPromotion() + useUpdatePromotion() + useRemovePromotion() + useGetMovies() + useAddMovie() + useUpdateMovie() + useRemoveMovie() + useCreateBranch() + useUpdateBranch() + useDeleteBranch()

These hooks handle system-wide administration tasks such as managing accounts, branches, movies, schedules, snacks, and promotions.

Functions:

useGetAccounts(): Retrieves the list of user accounts available in the system.

useAddAccount(): Adds a new user account with specified roles and credentials.

useUpdateAccount(): Updates details of an existing user account.

useUpdateUserPermission(): Updates roles for an existing user account.

useRemoveAccount(): Deletes a user account from the system.

useGetPromotions(): Retrieves the list of all promotions in the system.

useAddPromotion(): Adds a new promotion to be applied to tickets or snacks.

useUpdatePromotion(): Updates details of an existing promotion.

useRemovePromotion(): Deletes a promotion from the system.

useGetMovies(): Retrieves the list of movies available for administration.

useAddMovie(): Adds a new movie to the system with full metadata.

useUpdateMovie(): Updates the details of an existing movie.

useRemoveMovie(): Removes a movie from the system.

useAddBranch(): Adds a new cinema branch with its information.

useUpdateBranch(): Updates details of an existing branch.

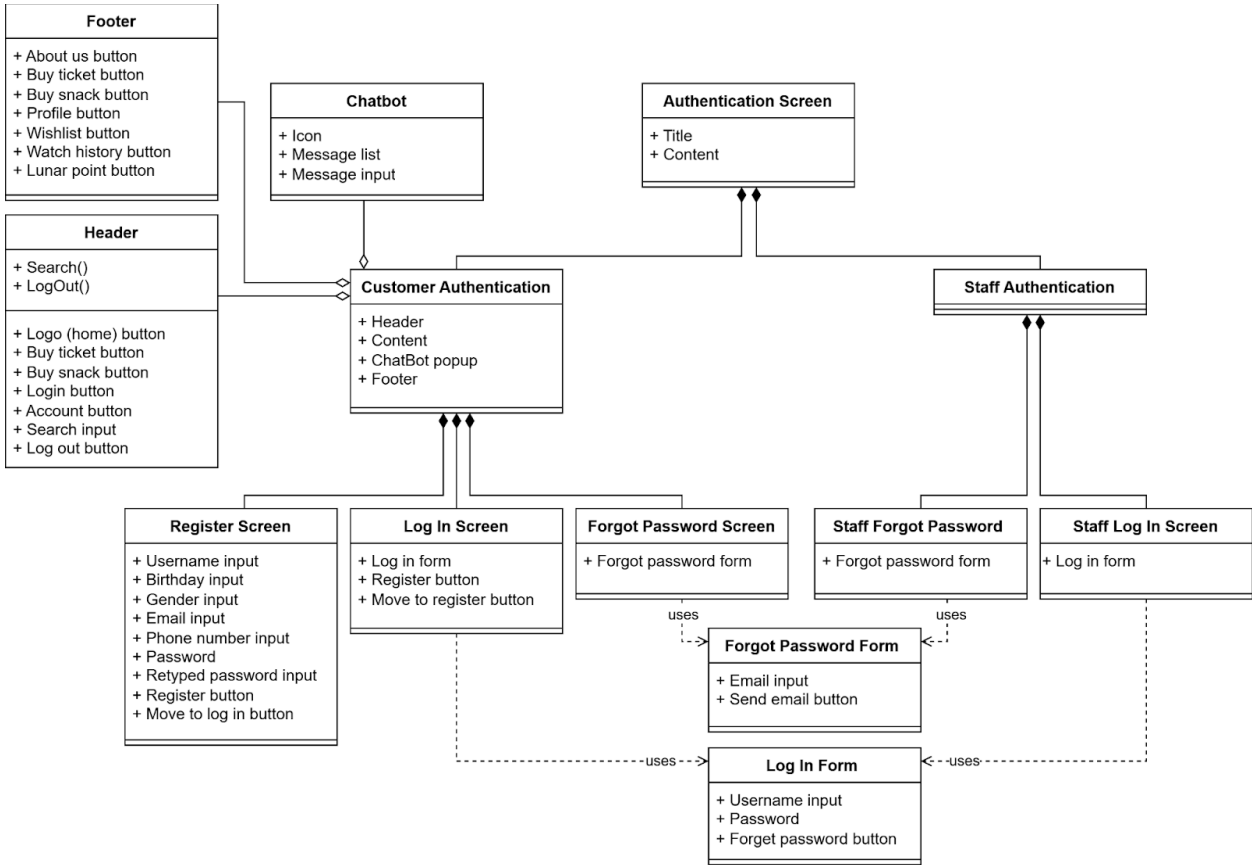
useRemoveBranch(): Removes a branch from the system.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.3 Component: User Interface (UI)

The UI component layer of Lumiere Cinema is structured around five main layouts: Authentication, Main, Ticket Booking, User Profile, and Admin Dashboard. Each layout encapsulates related screens and interactions, ensuring clear separation of concerns and consistent user experience across different roles. Components are designed to be modular and reusable, with shared elements like Header, Footer, and Chatbot integrated into the Main layout. Navigation flows logically between components – for example, users can browse movies, proceed to book tickets, and later access their booking history through the profile section. This architecture supports both customer and staff, providing a flexible and scalable interface that aligns with the underlying hook and service layers.

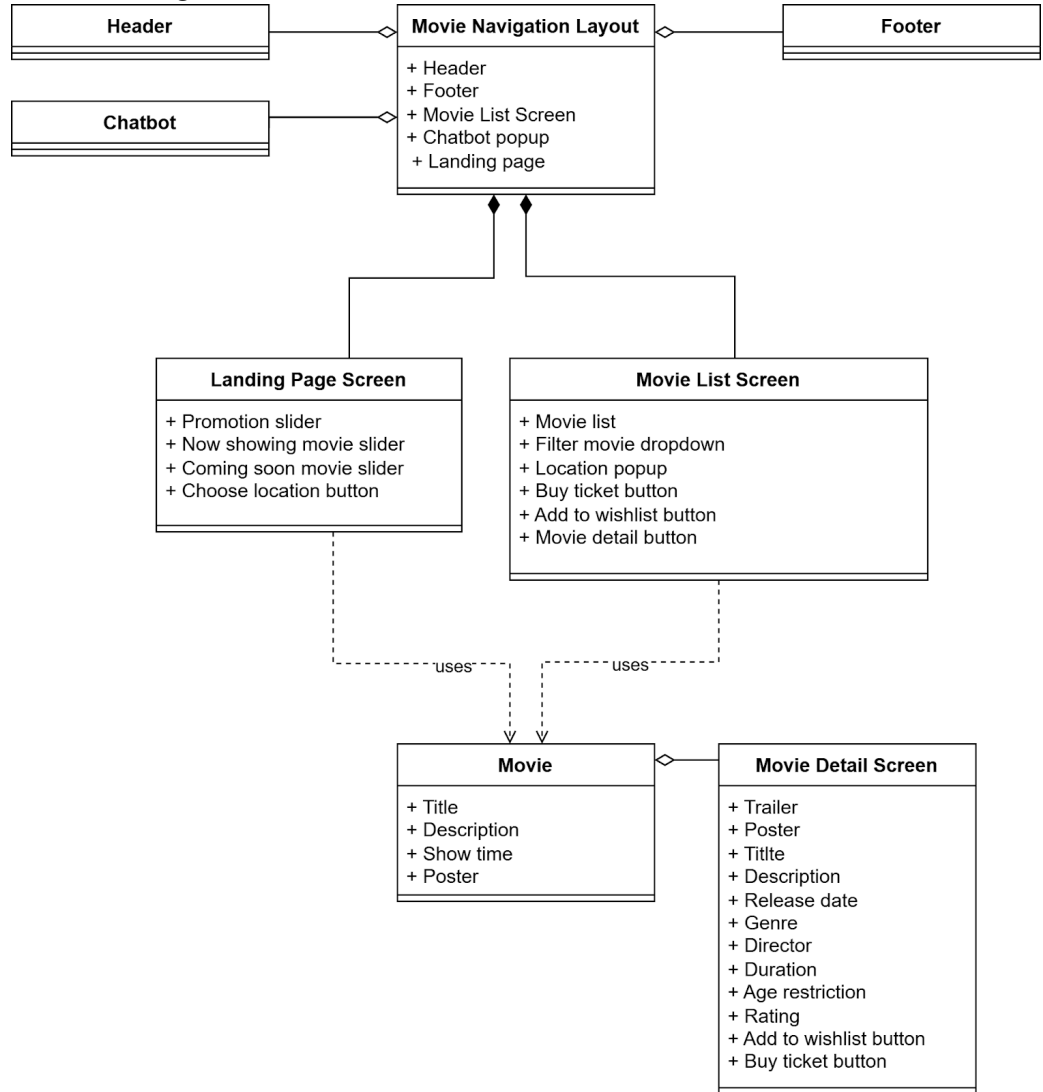
4.3.1 Authentication layout



This package manages user authentication, including registration, login, and password recovery screens. It distinguishes between staff and customer login flows: staff have access only to login and password reset, with a minimal UI optimized for fast interaction, while customers can register, log in, and reset passwords, and also navigate customer-specific screens via integrated header and footer components. The package ensures each user is properly authenticated and authorized to access the screens corresponding to their role.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.3.2 Movie navigation



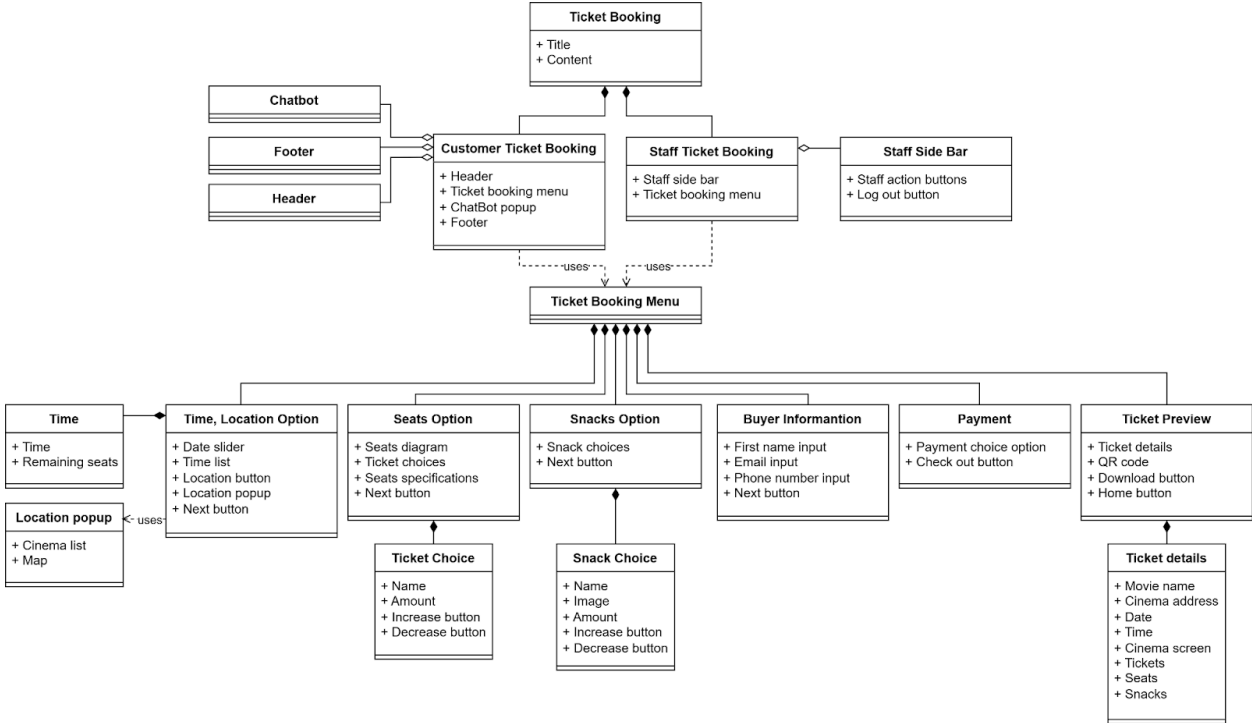
This diagram illustrates the user interface layout for the Movie Navigation Component, which supports user interaction with movie-related content and ticket purchasing features. The main structure is organized around the Movie Navigation Layout, which integrates reusable components such as Header, Footer, and Chatbot, along with dedicated movie navigation screens.

From the Movie Navigation Layout, users can access the following sub-screens:

- **Landing Page Screen:** Serves as the homepage for the movie portal. It includes a promotion slider, sections for now showing and coming soon movies, and allows users to select their preferred cinema location.
- **Movie List Screen:** Displays a scrollable list of movies. Users can filter movies, choose a cinema location, view movie details, add movies to their wishlist, or proceed to ticket booking.
- **Movie Detail Screen:** Shows comprehensive information about a selected movie, including trailer, poster, description, release date, genre, director, duration, rating, and age restriction. From here, users can also add the movie to their wishlist or buy a ticket.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

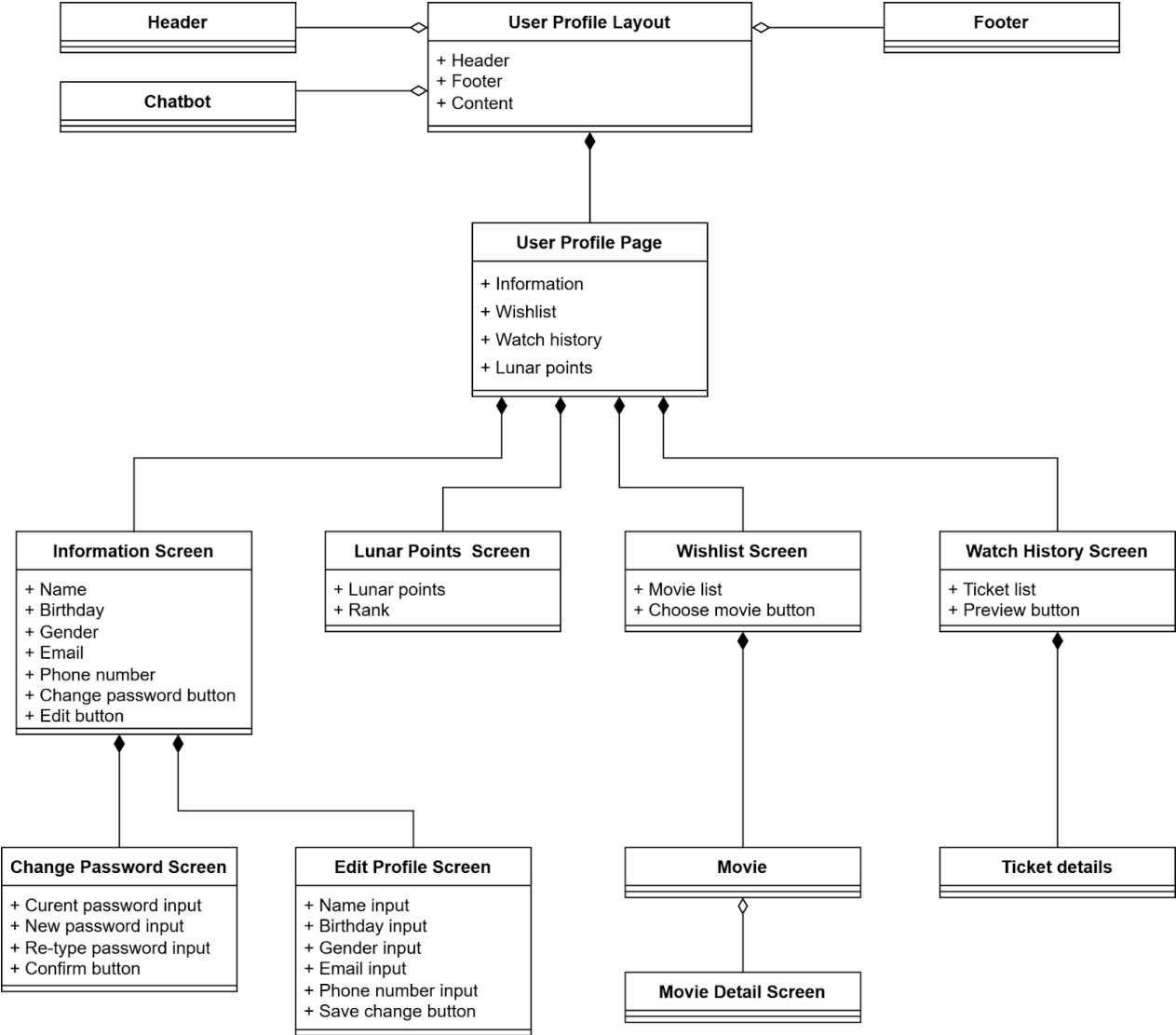
4.3.3 Ticket booking flow



This package manages the ticket booking process for both movie and snack tickets. The customer layout includes reusable components such as a header, footer, and chatbot popup, while the staff layout is simpler, featuring a sidebar for easy access to staff-specific functions. Despite the differences in layout, both customer and staff interfaces utilize the core component of this package: the Ticket Booking Menu. This menu guides users through a step-by-step process, including selecting time and location, choosing seats and snacks, entering buyer information, selecting a payment method, and finally, previewing the completed ticket.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.3.4 User profile



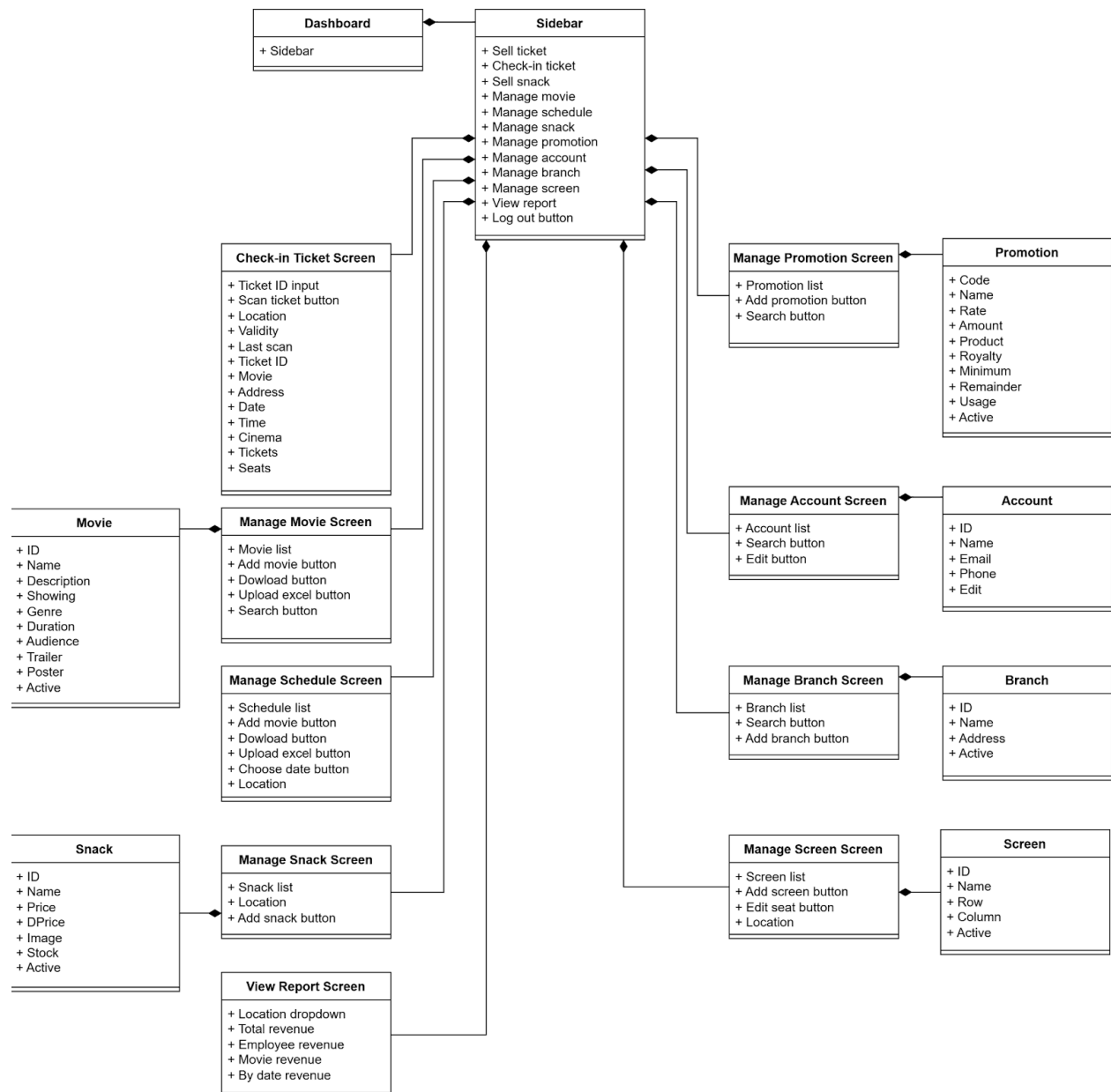
This diagram illustrates the user interface layout for the user profile section. The main component is `UserProfileLayout`, which includes reusable parts like `Header`, `Footer`, and the central content area: `User Profile Page`.

From the `User Profile Page`, users can access the following sub-screens:

- `Information Screen`: Displays personal details such as name, birthday, gender, email, and phone number. It also provides access to two subscreens:
 - + `Edit Profile Screen`: Allows users to update their personal information.
 - + `Change Password Screen`: Allows users to securely change their password.
- `Wishlist Screen`: Shows the user's favorite movies and allows selecting a movie.
- `Watch History Screen`: Lists previously viewed tickets with preview options.
- `Lunar Points Screen`: Displays the user's loyalty points and ranking.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.3.5 Dashboard layout



This diagram illustrates the user interface layout for the Dashboard Component of the cinema management system. The central component is the Dashboard, which includes a reusable Sidebar for navigation across various management and operational screens.

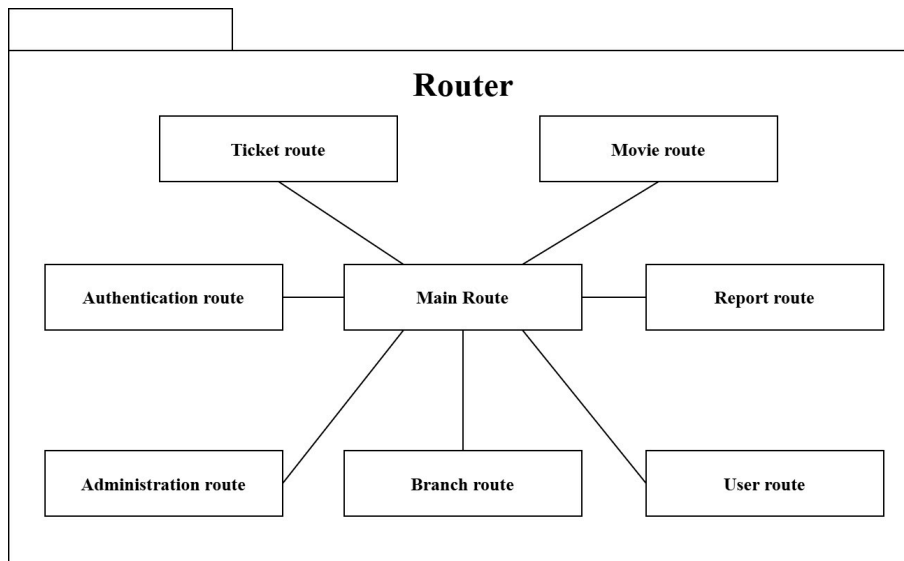
From the Sidebar, users can access the following sub-screens:

- Check-in Ticket Screen: Allows staff to scan ticket IDs, verify validity, and view details such as movie, date, time, and cinema location.
- Manage Movie Screen: Displays the list of available movies and provides functions to add, search, download, or upload movie data.
- Manage Schedule Screen: Enables administrators to manage screening schedules, with options to add movies, filter by location, and handle Excel uploads/downloads.
- Manage Snack Screen: Shows the snack list and allows filtering by cinema branch.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

- Manage Promotion Screen: Manages promotion codes and their attributes such as rate, product type, royalty, and usage statistics.
- Manage Account Screen: Displays user account information with features to search and edit account details.
- Manage Branch Screen: Allows administrators to manage cinema branches including their names, addresses, and statuses.
- Manage Screen Screen: Provides functionality to manage cinema screens (halls), including editing screen layout and filtering by branch.
- View Report Screen: Displays financial and operational statistics such as total revenue, revenue by employee, movie revenue, and filters for location and date.

4.4 Component: Routers



Main route is the designated place to apply middleware functions (CORS, Body Parsing,...) that are executed for all incoming requests across the system. After processing global middleware, the Main Route delegates requests to their corresponding specialized sub-routers based on their Base URL. This ensures that each type of resource (authentication, user, movie, branch, ticket, report, administration) is handled by its dedicated route module, promoting a well-structured and modular back-end architecture

For example: The Lumiere Cinema login begins when a user interacts with the UI Component, triggering its `login()` function. This calls the `useAuthentication` Hook, which then sends an HTTP POST request via the `AuthenticationService` to the back-end's `/api/auth/login` endpoint. Upon arrival, the Router recognizes `/api/auth/login` as an endpoint and directs it to the `Authentication Controller`.

4.4.1 Authentication route

Base URL: `/api/auth`

- **POST Routes:**
 - + `/login`: Authenticates user credentials (email and password). If valid, the server generates an Access Token and Refresh Token, then returns them along with user info to the client for session management.
 - + `/register`: Registers a new user account. The server securely hashes the password and stores the user's information in the database.
 - + `/logout`: Logs the user out of the system. The server invalidates the current refresh token.
 - Requires Authentication: Customer, Cashier, Checkin Counter, Branch Manager, Administrator.
 - + `/reset-password`: Initiates a password reset process. The server generates a secure reset token and sends a password reset link to the user's email address.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

- + **/change-password**: Allows the currently authenticated user to change their password. The server verifies the current password before updating to the new one to ensure security.
 - Requires Authentication: Customer, Cashier, Checkin Counter, Branch Manager, Administrator.

4.4.2 User route

Base URL: /api/users

- **GET Routes:**
 - + **/me**: Get the profile details of the currently authenticated user.
 - Requires Authentication: Customer
 - + **/tickets**: Get a list of movie tickets booked by the currently authenticated user.
 - Requires Authentication: Customer
 - + **/wishlist**: Retrieves the list of movies the authenticated user has added to their wishlist.
 - Requires Authentication: Customer
 - + **/rated**: Retrieves the all rating that the currently authenticated user has given to movies.
 - Requires Authentication: Customer
- **POST Routes:**
 - + **/wishlist**: Adds a specified movie to the authenticated user's wishlist.
 - Requires Authentication: Customer
 - + **/rate/:movieId**: Submits a new rating for a movie that the authenticated user has previously watched and is eligible to rate.
 - Requires Authentication: Customer
 - + **/query**: Submits a question or query to the AI chatbot and receives a response.
 - Requires Authentication: Customer
- **PUT Routes:**
 - + **/me**: Fully updates the profile details of the currently authenticated user. All editable fields must be provided.
 - Requires Authentication: Customer
- **PATCH Routes:**
 - + **/me**: Performs a partial update on the profile details of the currently authenticated user. Only the fields provided in the request body will be modified.
 - Requires Authentication: Customer
- **DELETE Routes:**
 - + **/wishlist/:movieId**: Removes a specific movie from the authenticated user's wishlist.
 - Requires Authentication: Customer

4.4.3 Movie route

Base URL: /api/movies

- **GET Routes:**
 - + **/now-showings**: Retrieves a list of movies that are currently being shown.
 - Requires Authentication: Customer, Administrator
 - + **/upcoming**: Retrieves a list of upcoming movies.
 - Requires Authentication: Customer, Administrator
 - + **/search**: Searches for movies based on keywords or criteria like genre.
 - Requires Authentication: Customer
 - + **/:movieId**: Retrieves detailed information for a specific movie, including synopsis, genre, trailer, and screening schedules.
 - Requires Authentication: Customer
 - + **/:movieId/showscreen**: Retrieves all screenings schedules for a given movie.
 - Requires Authentication: Customer
 - + **/all**: Retrieves a comprehensive list of all movies in the system for management purposes.
 - Requires Authentication: Administrator
 - + **/:movieId/get-ratings**: Retrieves rating of a specific movie.
 - Requires Authentication: Customer

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

- **POST Routes:**
 - + **/:** Adds a new movie to the system, including its information (title, genre, duration, etc.).
 - Requires Authentication: Administrator
- **PUT Routes:**
 - + **/:movieId:** Fully updates all details of an existing movie.
 - Requires Authentication: Administrator
- **PATCH Routes:**
 - + **/:movieId:** Partially updates specific fields of an existing movie's information.
 - Requires Authentication: Administrator
- **DELETE Routes:**
 - + **/:movieId:** Removes a movie from the system or deactivates it from public display.
 - Requires Authentication: Administrator

4.4.4 Branch route

Base URL: [/api/branches](#)

- **GET Routes:**
 - + **/all:** Retrieves a list of all branches.
 - Requires Authentication: Administrator
 - + **/:branchId/screens:** Retrieves a list of screens/halls available at a specific branch.
 - Requires Authentication: Branch Manager
 - + **/:branchId/screens/:screenId:** Retrieves detailed information for a specific screen within a branch for management purposes.
 - Requires Authentication: Branch Manager
 - + **/:branchId/snacks:** Retrieves the list of available snack combos and individual snack items for a specific cinema branch
 - Requires Authentication: Branch Manager
 - + **/:branchId/snacks/:snackId:** Retrieves detailed information for a specific snack item within a branch for management purposes.
 - Requires Authentication: Branch Manager
 - + **/:branchId/schedules:** Retrieves all movie screening schedules for a specific branch, optionally filtered by date.
 - Requires Authentication: Branch Manager
 - + **/:branchId/schedules/detail:** Retrieves detailed information for a specific movie schedule within a branch for management purposes.
 - Requires Authentication: Branch Manager
 - + **/:branchId/schedules/available-seats:** Retrieves the seats layout of schedule with state of seats(available, unavailable).
 - Requires Authentication: Branch Manager
- **POST Routes:**
 - + **/new-branch:** Adds a new cinema branch to the system.
 - Requires Authentication: Administrator
 - + **/:branchId/screens:** Creates a new cinema screen within a specific branch..
 - Requires Authentication: Branch Manager
 - + **/:branchId/snacks:** Adds a new snack item to a specific branch's snack list.
 - Requires Authentication: Branch Manager
 - + **/:branchId/schedules:** Sets a new movie screening at a specific screen and time for a branch.
 - Requires Authentication: Branch Manager
- **PUT Routes:**
 - + **/:branchId:** Fully updates all high-level details of an existing cinema branch.
 - Requires Authentication: Administrator
 - + **/:branchId/screens/:screenId:** Fully updates details of an existing screen within a specific branch.
 - Requires Authentication: Branch Manager

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

- + **/:branchId/snacks/:snackId**: Fully updates details of an existing snack item for a specific branch..
 - Requires Authentication: Branch Manager
- + **/:branchId/schedule-detail**: Fully updates the movie screening at a specific screen and time for a branch.
 - Requires Authentication: Branch Manager
- **PATCH Routes:**
 - + **/:branchId**: Partially updates high-level details of an existing cinema branch.
 - Requires Authentication: Administrator
 - + **/:branchId/screens/:screenId**: Partially updates details of an existing screen within a specific branch.
 - Requires Authentication: Branch Manager
 - + **/:branchId/snacks/:snackId**: Partially updates details of an existing snack item for a specific branch.
 - Requires Authentication: Branch Manager
 - + **/:branchId/schedule-detail**: Partially modifies details of an existing movie schedule for a specific branch.
 - Requires Authentication: Branch Manager
- **DELETE Routes:**
 - + **/:branchId**: Removes a cinema branch from the system.
 - Requires Authentication: Administrator
 - + **/:branchId/screens/:screenId**: Removes a specific screen from a specific branch.
 - Requires Authentication: Branch Manager
 - + **/:branchId/snacks/:snackId**: Removes a snack item from a specific branch's list.
 - Requires Authentication: Branch Manager
 - + **/:branchId/schedule-detail**: Cancels an existing movie schedule for a specific branch.
 - Requires Authentication: Branch Manager

4.4.5 Ticket route

Base URL: /api/tickets

This API section manages all operations related to movie tickets and snack purchases within the Lumiere Cinema system. It provides functionalities for users to view their purchased tickets and for authorized personnel (Customers or Cashiers for purchase, Admins for full management) to interact with ticket data.

Movie Ticket Routes: /api/tickets/movies

This subset of routes specifically handles movie ticket functionalities, linking users to their booked seats for specific movie schedules.

- **POST Routes:**
 - + **/**: Purchases one or more movie tickets for specific seats and schedules.
 - Allows a Customer or Cashier to book seats for a movie screening. The request body will contain details like scheduleId, seatId(s), and may contain customerId and/or staffId.
 - Requires Authentication: Customer, Cashier
- **GET Routes:**
 - + **/me**: Retrieves a list of all movie tickets purchased by the currently authenticated user.
 - Provides a historical record of movie tickets for the logged-in customer.
 - Requires Authentication: Customer
 - + **/me/:ticketId**: Retrieves detailed information for a specific movie ticket belonging to the currently authenticated user.
 - Allows a customer to view details of an individual movie ticket.
 - Requires Authentication: Customer
 - + **/admin/all**: Retrieves a comprehensive list of all movie tickets in the system.
 - Designed for administrative purposes to view all movie ticket transactions.
 - Requires Authentication: Branch Manager, Administrator

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

- + **/admin/:ticketId**: Retrieves detailed information for any specific movie ticket by its ID.
 - Allows administrators to inspect details of any individual movie ticket.
 - Requires Authentication: Cashier, Checkin Counter, Branch Manager, Administrator.
- **PATCH Routes:**
 - + **/admin/:ticketId**: Performs a partial update on the details of an existing movie ticket. For administrative control, allows modification of ticket properties, especially status changes. Customers cannot modify ticket status.
 - Requires Authentication: Cashier, Checkin Counter, Branch Manager, Administrator.
- **DELETE Routes:**
 - + **/admin/:ticketId**: Cancels or logically deletes a specific movie ticket from the system. For administrative control, used for cancellations or refunds.
 - Requires Authentication: Branch Manager, Administrator.

Snack Ticket Routes: /api/tickets/snacks

This subset of routes specifically handles snack ticket functionalities, detailing the purchase of food and beverage items.

- **POST Routes:**
 - + **/**: Purchases one or more snack items.
 - Allows a Customer or Cashier to purchase snack products. The request body will contain details like snackId, quantity, and may contain customerId and/or staffId.
 - Requires Authentication: Customer or Cashier
- **GET Routes:**
 - + **/me**: Retrieves a list of all snack tickets purchased by the currently authenticated user.
 - Provides a historical record of snack purchases for the logged-in customer.
 - Requires Authentication: Customer
 - + **/me/:ticketId**: Retrieves detailed information for a specific snack ticket belonging to the currently authenticated user.
 - Description: Allows a customer to view details of an individual snack ticket.
 - Requires Authentication: Customer
 - + **/admin/all**: Retrieves a comprehensive list of all snack tickets in the system.
 - Description: Designed for administrative purposes to view all snack ticket transactions.
 - Requires Authentication: Branch Manager, Administrator
 - + **/admin/:ticketId**: Retrieves detailed information for any specific snack ticket by its ID.
 - Description: Allows administrators to inspect details of any individual snack ticket.
 - Requires Authentication: Cashier, Checking Counter, Branch Manager, Administrator.
- **PATCH Routes:**
 - + **/admin/:ticketId**: Performs a partial update on the details of an existing snack ticket
 - For administrative control, allows modification of ticket properties, especially status changes. Customers cannot modify ticket status.
 - Requires Authentication: Cashier, Checkin Counter, Branch Manager, Administrator.
- **DELETE Routes:**
 - + **/admin/:ticketId**: Cancels or logically deletes a specific snack ticket from the system.
 - For administrative control, used for cancellations or refunds.
 - Requires Authentication: Branch Manager, Administrator.

4.4.6 Report route

Base URL: /api/reports

- **GET Routes:**
 - + **/revenue**: Retrieves a comprehensive revenue report, with filtering options for time periods and branches.
 - Requires Authentication: Administrator
 - + **/sales-by-movie**: Retrieves a report on sales performance per movie.
 - Requires Authentication: Administrator

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.4.7 Administration route

Base URL: `/api/admin`

- GET Routes:

- + `/users/all`: Retrieves a comprehensive list of all user accounts within the system, along with key details and management options for each account.
 - Requires Authentication: Administrator
- + `/users/:userId`: Retrieves detailed profile information for a specific user account.
 - Requires Authentication: Administrator
- + `/promotions/all`: Retrieves a comprehensive list of all promotional campaigns (including inactive or expired ones) for management purposes.
 - Requires Authentication: Administrator
- + `/promotions/:promoId`: Retrieves a promotional campaign.
 - Requires Authentication: Administrator

- POST Routes:

- + `/users`: Adds a new user account.
 - Requires Authentication: Administrator
- + `/promotions`: Adds a new promotional campaign to the system.
 - Requires Authentication: Administrator

- PUT Routes:

- + `/users/:userId`: Fully updates the profile details of any user account.
 - Requires Authentication: Administrator
- + `/promotions/:promoId`: Fully updates all details of an existing promotional campaign.
 - Requires Authentication: Administrator

- PATCH Routes:

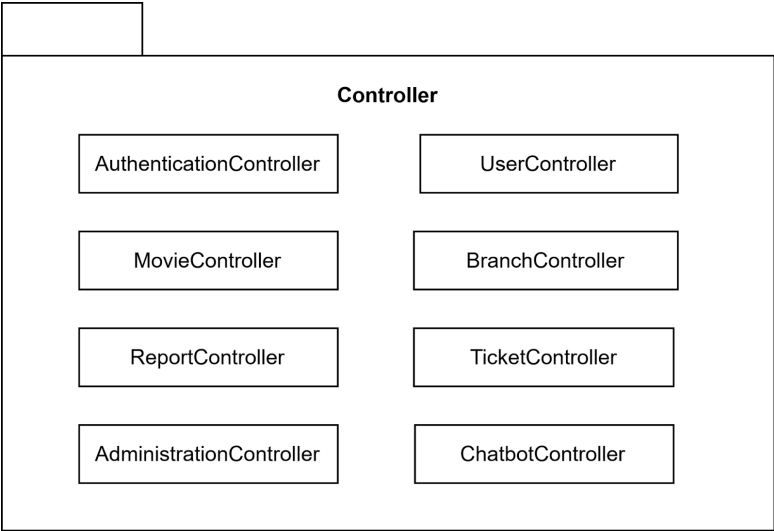
- + `users/:userId`: Partially updates the profile details of any user account.
 - Requires Authentication: Administrator
- + `/promotions/:promoId`: Partially updates specific fields of an existing promotional campaign.
 - Requires Authentication: Administrator
- + `/users/:userId/permissions`: Manages the roles assigned to a specific user account by adding or removing roles from a list of available options.
 - Requires Authentication: Administrator

- DELETE Routes:

- + `/users/:userId`: Deletes a user account from the system.
 - Requires Authentication: Administrator
- + `/promotions/:promoId`: Remove a promotional campaign.
 - Requires Authentication: Administrator

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.5 Component: Controllers



These controllers are responsible for receiving requests from routes, parsing them, and processing them within the Model component. Controllers also format responses, ensuring data is returned consistently and handling HTTP status codes, thereby maintaining a clear interface between clients and the back-end.

4.5.1 Authentication controller

AuthenticationController	login () : Calls userModel.authenticate () to verify credentials. If successful, generates and returns tokens.
- userModel: UserModel	register () : Calls userModel.createOne () to register a new user after securely hashing the password.
+ login() + register() + logout() + resetPassword() + changePassword()	logout () : Invalidates the current refresh token. resetPassword () : Generates a reset token and triggers an email to the user with a password reset link. changePassword () : Verifies the old password and calls userModel.changePassword () to update it.

4.5.2 User controller

UserController	getProfile () :Retrieves the profile of the currently authenticated user. Uses UserModel.getOne () or find () .
- userModel: UserModel - ticketModel: TicketModel - movieModel: MovieModel	updateProfile () : Performs update of user info using UserModel.updateOne () .
+ getProfile() + updateProfile() + getMyTickets() + getWishlist() + addToWishlist() + removeFromWishlist() + rateMovie()	getMyTickets () : Retrieves all tickets booked by the user. Requires interaction with TicketModel. getWishlist () : Retrieves movies saved to the user's wishlist via Customer.wishlist . addToWishlist () : Adds a movie to the wishlist using Customer.addToWishlist () . removeFromWishlist () : Removes a movie from wishlist using Customer.removeFromWishlist () . rateMovie () : Submits a rating using MovieModel.rateMovie () .

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.5.3 Movie controller

MovieController
-movieModel: MovieModel -screenModel: ScreenModel
+getNowShowingMovies() +getUpcomingMovies() +getMovieDetails() +getAllMovies() +addMovie() +updateMovie() +deleteMovie() +getMovieShowtimes() +searchMovies() +getMovieRatingsSummary()

Manages the retrieval, creation, updating, and deletion of movie data, including detailed information, search functionality, user ratings, and administrative controls.

Functions:

getNowShowingMovies () : Retrieve a list of now showing movies.
getUpcomingMovies () : Retrieve a list of upcoming movies.
getMovieDetails () : Retrieve comprehensive details for a specific movie, including its trailer, synopsis, genre,...
getAllMovies () : Retrieves a comprehensive list of all movies in the system for management purposes.
addMovie () : Adds a new movie to the system.
updateMovie () : Updates an existing movie.
deleteMovie () : Deletes an existing movie.
getMovieShowtimes () : Retrieves specific screening schedules for a given movie.

searchMovie () : Enables users to locate relevant movies by specifying search criteria.

getMovieRatingSummary () : Retrieves specific rating of an existing movie.

4.5.4 Branch controller

Manages operations related to cinema branches, including screen configurations, snack lists, and movie screening schedules.

Functions:

createScreen () : Create a new cinema screen within their assigned branch.
editScreen () : Modify details of an existing screen.
deleteScreen : Remove a screen from a specific branch.
configureScreenLayout () : Set up seat layouts and pricing for a specific screen.
createSnack () : Add a new snack item to their branch's snack list.
editSnack () : Modify details of an existing snack item.
deleteSnack () : Remove a snack item from their branch's list.
getSnackList () : Retrieves the list of available snack items for a specific branch.
scheduleMovieScreening () : Set a new movie screening at a specific screen and time.
editMovieSchedule () : Modify details of an existing movie schedule.
deleteMovieSchedule () : Cancel an existing movie schedule.
getMovieSchedules () : Retrieves lists of movie schedules.

BranchController
-branchModel: BranchModel -screenModel: ScreenModel -snackModel: SnackModel
+createScreen() +editScreen() +deleteScreen() +configureScreenLayout() +createSnack() +editSnack() +deleteSnack() +getSnackList() +scheduleMovieScreening() +editMovieSchedule() +deleteMovieSchedule() +getMovieSchedules()

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.5.5 Report controller

ReportController
-ticketModel: TicketModel -movieModel: MovieModel -branchModel: BranchModel -screenModel: ScreenModel
+getRevenueReport() +getSalesByMovieReport() +exportReport()

Manages requests related to generating and retrieving various operational and financial reports within the Lumiere Cinema system. It directly handles the logic for data retrieval, processing, and formatting required for revenue, sales, and occupancy reports.

Functions:

getRevenueReport(): Retrieves comprehensive revenue data based on filters

getSalesByMovieReport(): Retrieves sales performance data per movie, showing insights into movie popularity and revenue contribution.

exportReport(): Generates and provides a download link for a requested report in a specific format, based on provided report type and filters.

4.5.6 Ticket controller

TicketController
+ userModel: UserModel + ticketModel: TicketModel
+ createTicket() + getTicketById() + getTicketListByTime() + checkInTicket() + makeTicketValid() + updateTicket() + getTotal() + callLunarPoints() - generateTicketCode()

Manage operations related to tickets, including creation, data retrieval, validation, and updates of ticketModel instances. It supports ticket booking, check-in, and provides ticket data for report generation.

createTicket(): Create a new ticket based on the appropriate TicketModel type, with input validation.

getTicketById(): Return the TicketModel instance that matches the provided ID.

getTicketListByTime(): Return a list of tickets whose creation date falls within the specified time range.

checkInTicket(): Validate and record the ticket check-in date. Checks for validity, prior check-in, and expiration.

makeTicketValid(): Set the status of the ticket to valid, if it was previously invalid.

updateTicket(): Update the ticket's details with the provided data.

getTotal(): Return the total cost of the selected ticket (including promotions or additional items).

callLunarPoints(): Return the total point obtained upon purchasing the ticket.

generateTicketCode(): Generate a unique, random code for identifying the ticket.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.5.7 Administration controller

Manages high-level administrative tasks and system configurations in the Lumiere Cinema system. It handles users, promotions, and branches, with all functions restricted to users with the Administrator role.

Functions:

getAllUsers(): Retrieves a comprehensive list of all user accounts within the system, along with key details for management.

getUserDetails(): Retrieves detailed profile information for a specific user account.

updateUserRoles(): Manages the roles assigned to a specific user account, allowing for adding or removing roles.

updateUserStatus(): Changes the activation status of a user account.

deleteUser(): Deletes a user account from the system.

getAllPromotions(): Retrieves a comprehensive list of all promotional campaigns, including inactive or expired ones.

createPromotion(): Creates a new promotional campaign in the system.

updatePromotion(): Updates an existing promotional campaign.

deletePromotion(): Deactivate or delete a promotional campaign from the system.

getAllBranches(): Retrieves a comprehensive list of all cinema branches for overall management.

AdministratorController
- userModel: UserModel - promotionModel: PromotionModel - snackModel: SnackModel - branchModel: BranchModel
+ getAllUsers() + getUserDetails() + updateUserRoles() + updateUserStatus() + deleteUser() + getAllPromotions() + createPromotion() + updatePromotion() + deletePromotion() + getAllBranches() + createBranch() + updateBranch() + deleteBranch() + deactivateBranch()

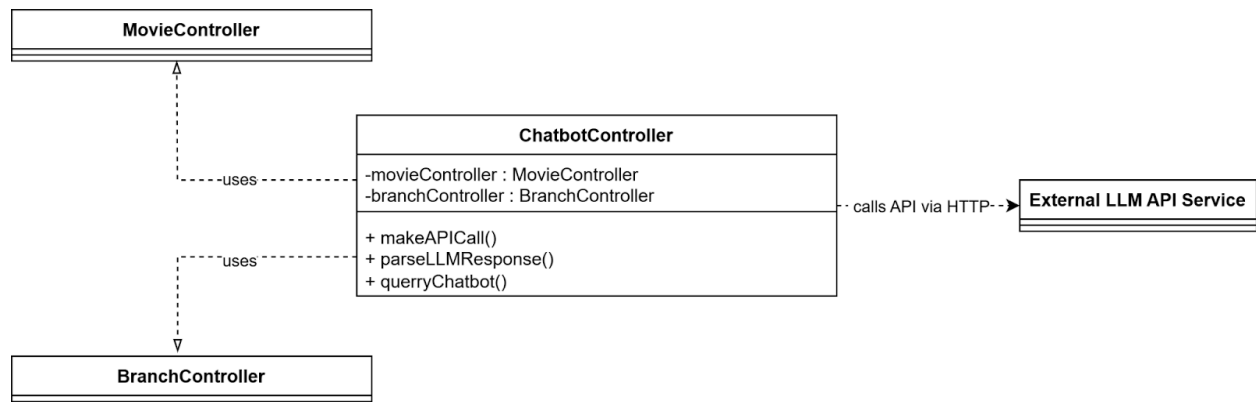
createBranch(): Creates a new cinema branch in the system.

updateBranch(): Updates an existing cinema branch

deleteBranch(): Deletes a cinema branch from the system.

updateBranchStatus(): Deactivates a specific cinema branch, making it inactive in the system.

4.5.8 Chat-bot controller



Acts as an intermediary, enabling users to interact with an AI chatbot for movie-related inquiries using natural language. It interprets user queries, communicates with the Large Language Model (LLM) to understand intent and extract information, and then invokes relevant functions in other internal controllers to provide accurate responses.

Functions:

queryChatbot(): Processes natural language queries from the user, forwards them to the LLM service for analysis, and then orchestrates calls to appropriate functions in other controllers (like MovieController or BranchController) based on the LLM's understanding of the user's intent.

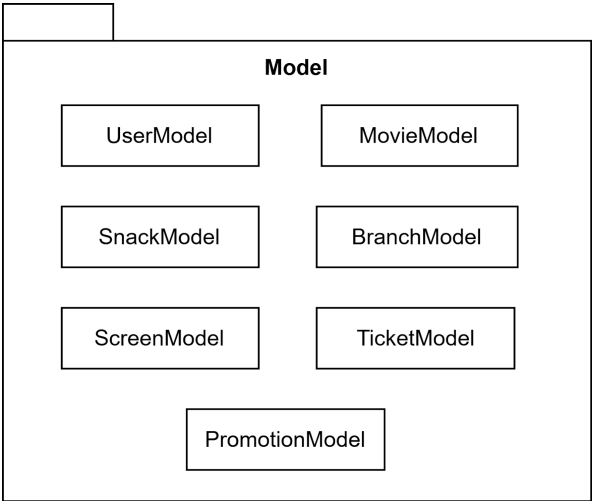
Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

makeLLMApiCall(): Makes the actual HTTP API call to the external Large Language Model (LLM) service, sending the user's query and handling the communication protocol.

parseLLMResponse(): Parses and extracts structured information (like identified intent and relevant entities) from the raw response received from the LLM service.

4.6 Component: Model

The Model component encapsulates the application's domain knowledge, managing the data and defining the operations that manipulate this data. This component acts as the central repository for the application's state and behavior, ensuring data integrity and consistency across all functionalities.

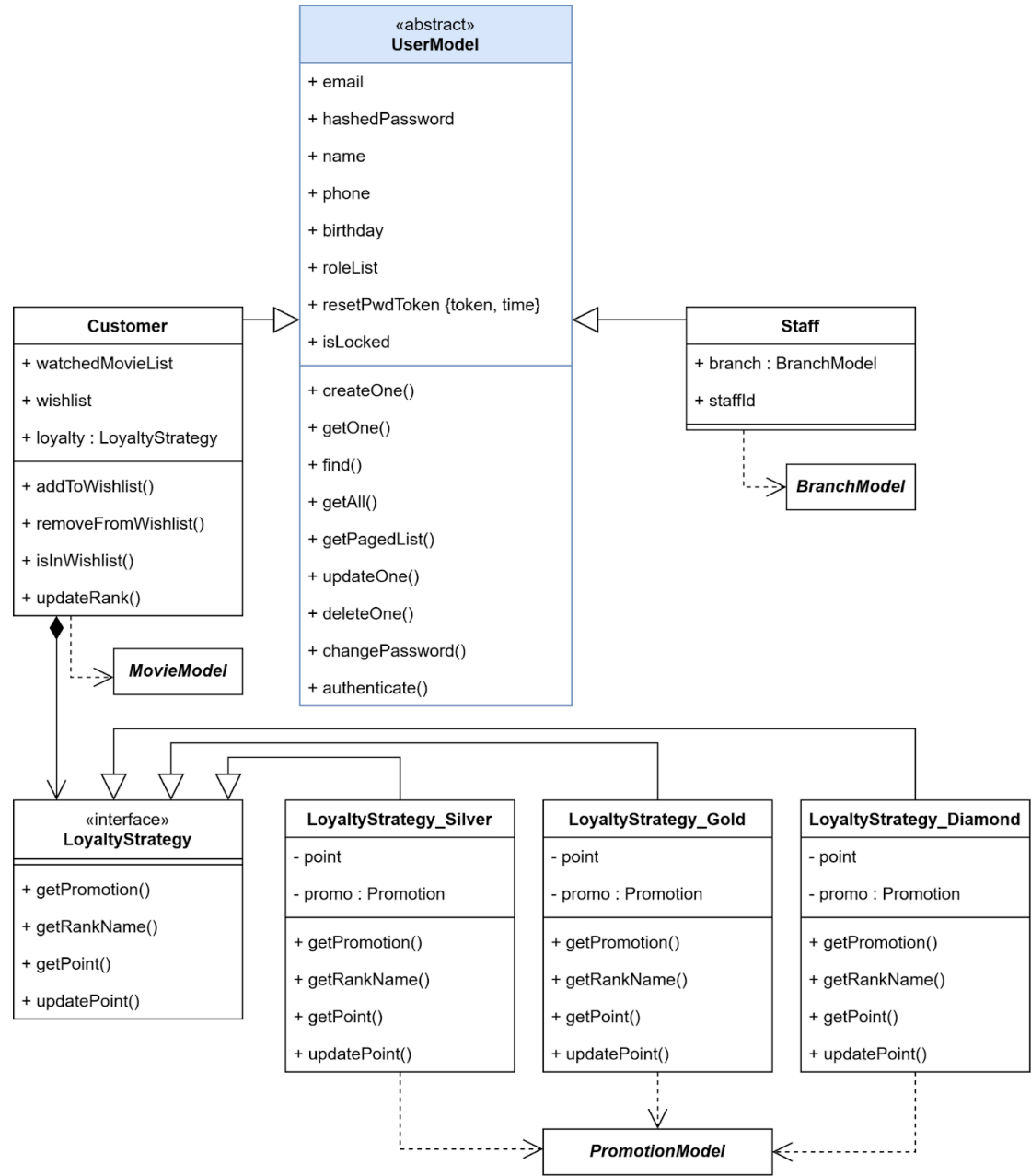


There are 7 main models, each model has its own purposes and features.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.6.1 UserModel

The UserModel is a critical part of the system's core, responsible for managing all user-related data and functionalities. It defines the structure of different user types and encapsulates the logic for user authentication, profile management, and loyalty program interactions. This model ensures a secure and personalized experience for both customers and internal staff.

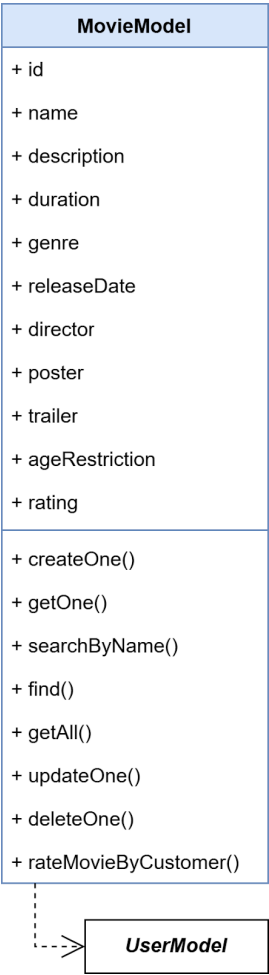


The common contract for various loyalty program tiers is also represented in the model using Strategy Pattern.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.6.2 *MovieModel*

MovieModel serves as the central entity for managing all information related to movies available in the Lumiere Cinema system. It encapsulates the complete set of data necessary to describe a movie to users and for internal management.



Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

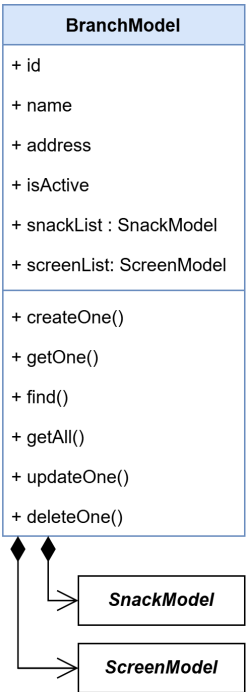
4.6.3 *SnackModel*

The *SnackModel* is designed to represent and manage the various food and beverage items offered for sale at Lumiere Cinema branches. It encapsulates the core details of each snack product, including its pricing, current availability (stock), and general information.



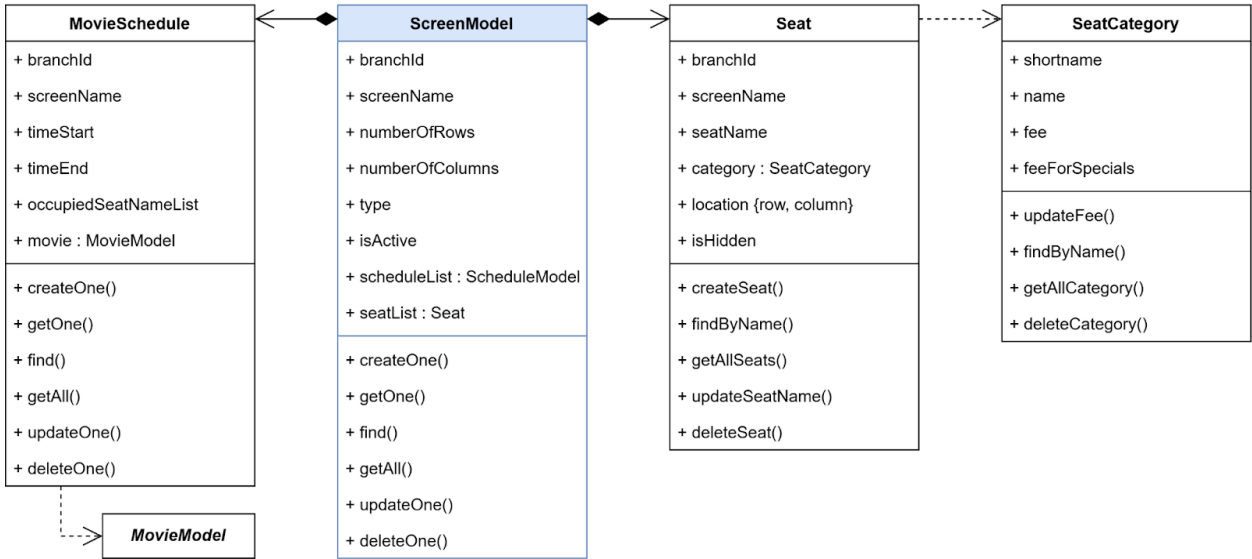
4.6.4 *BranchModel*

The *BranchModel* serves as a central entity representing a physical cinema location of Lumiere Cinema. It encapsulates the core administrative and operational details specific to each branch, acting as a container for its screens and the snack products it offers.



Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.6.5 ScreenModel

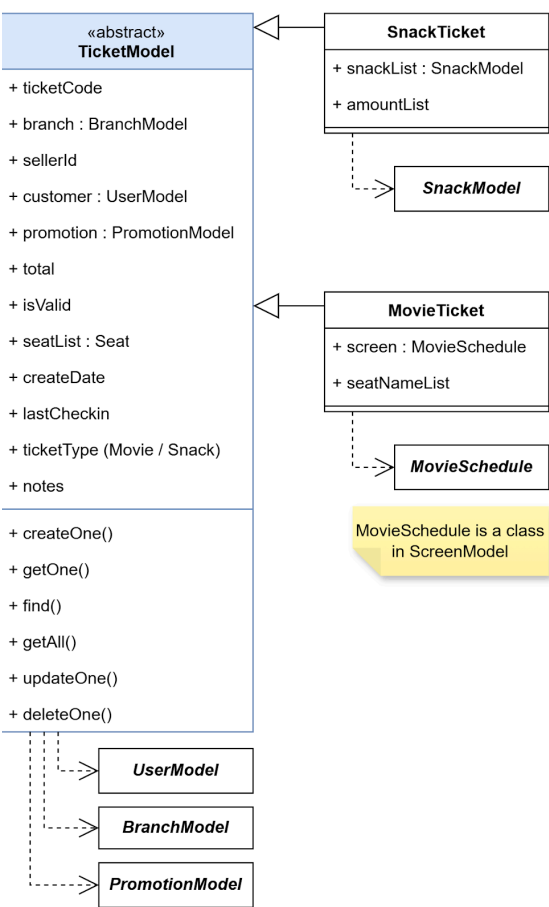


The **ScreenModel** class represents an individual movie screening room located within a cinema branch. It encapsulates the physical and operational characteristics of each screen, serving as a container for its seating layout and a key component in scheduling movie showtimes.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

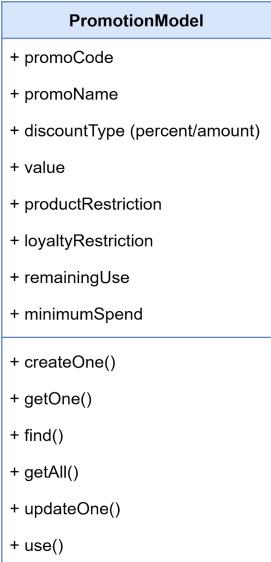
4.6.6 TicketModel

The TicketModel class within the Model component is a core entity that represents a single purchased item in the Lumiere Cinema system. This can be a ticket for a specific movie seat at a particular showtime, or it can be a ticket for a snack item. It encapsulates the details of the transaction and the status of the purchased item.



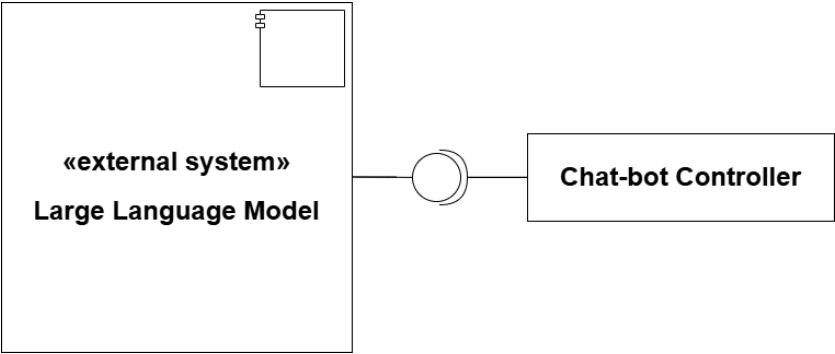
4.6.7 PromotionModel

The PromotionModel class within the Model component is designed to manage all information regarding promotional offers and discount campaigns within the Lumiere Cinema system. It encapsulates the details defining various types of promotions and their applicability, playing a crucial role in the system's marketing and loyalty programs.



Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

4.8 Component: Large Language Model



Purpose: The Large Language Model (LLM) component is integrated into the Lumiere Cinema system to enable natural language understanding (NLU) and generation (NLG) capabilities for the AI-based chatbot. Its primary purpose is to process user queries in natural language, understand their intent, extract relevant information, and assist in generating intelligent responses. In Lumiere Cinema context, the LLM is an external, cloud-based API service that the Lumiere Cinema back-end interacts with via HTTP/HTTPS. It is not hosted within Lumiere's direct infrastructure.

Integration Point: The Chat-bot Controller (located within the Controllers component) serves as the primary integration point with the LLM component .

Data Flow: The Chat-bot Controller receives user queries, forwards them to the LLM for analysis, and then orchestrates calls to other internal controllers (like Movie Controller or Branch Controller) based on the LLM's parsed intent .

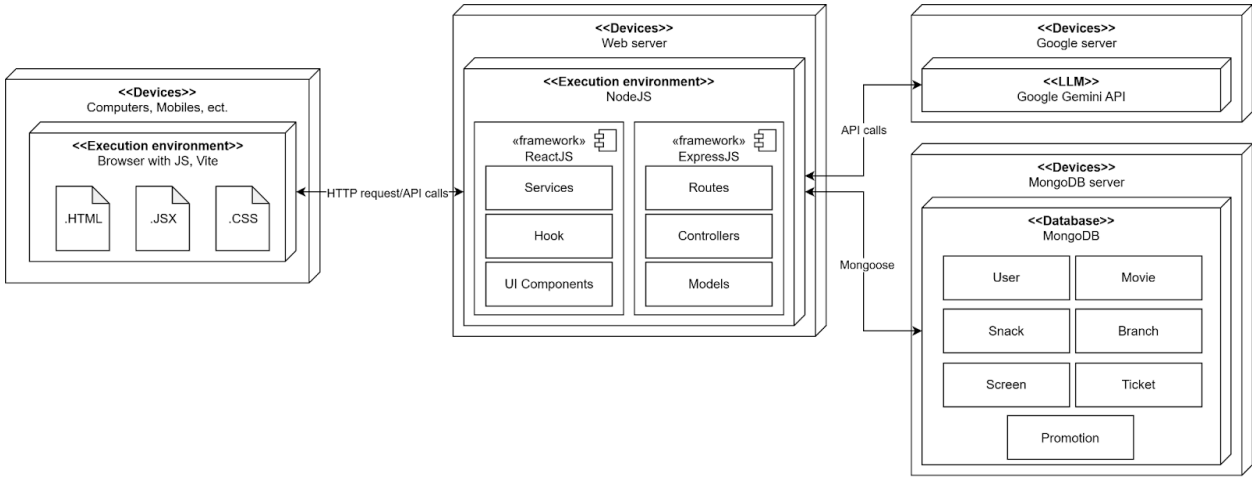
Communication Protocol: Communication between the Controllers component (specifically Chat-bot Controller) and the LLM component occurs over HTTP.

Key Technologies and Services:

- LLM Provider: (e.g., Google Gemini API).
- Internal Wrapper/Service: A dedicated service or wrapper within the Chat-bot Controller's logic handles direct API calls to the external LLM, abstracting its complexity .

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

5. Deployment



5.1 Client-side

The client-side of the Lumiere Cinema system consists of the user interface (UI) and is designed for broad accessibility and a consistent user experience across various devices and browsers.

- **Deployment:** The UI, developed using ReactJS and TailwindCSS, is accessed directly by end-users via standard web browsers. There is no dedicated client-side application server for distribution; the web application files are served by the Frontend Hosting/Web Server and rendered on the user's device.
- **Specifications:**
 - + *Supported Devices:* Desktops, Laptops (Windows, macOS), Tablets, and Smartphones (Android, iOS).
 - + *Supported Web Browsers:* Chromium-based browsers, Firefox and Apple's Safari on mobile. Any third-party services or APIs used by the client-side are also expected to support these browsers.

5.2 Server-side

The server-side encompasses the core business logic, data management, and API endpoints for the Lumiere Cinema application, built on a JavaScript runtime environment and a NoSQL database.

- **Deployment:**
 - + *Application Server:* All back-end services, hooks, routers, and controllers (including the Chat-bot Controller) are deployed on dedicated application servers, typically running Node.js with Express.js.
 - + *Database Server:* The MongoDB database can be deployed either on a dedicated local server or utilize a cloud database service like MongoDB Atlas.
 - + These application and database servers form the central backend infrastructure, handling API requests from the client-side and managing data persistence.
- **Specifications:**
 - + *Operating System:* Linux-based distributions are typically used for production environments.
 - + *Hardware/Resources:* Server specifications (CPU, RAM, Storage) will be scaled horizontally (via load balancing and multiple instances) to ensure high performance and maintain responsiveness within 3 seconds for 95% of cases, even during peak traffic.
 - + *Database Configuration:* For high availability, data integrity, and scalability, the database will be configured with replication (e.g., replica sets) and potentially sharding.

5.3 Third-Party Services Integration - Large Language Model (LLM)

The Large Language Model (LLM) component is integrated into the Lumiere Cinema system to enable intelligent, conversational capabilities within the AI-based chatbot. It enhances the user experience by enabling natural

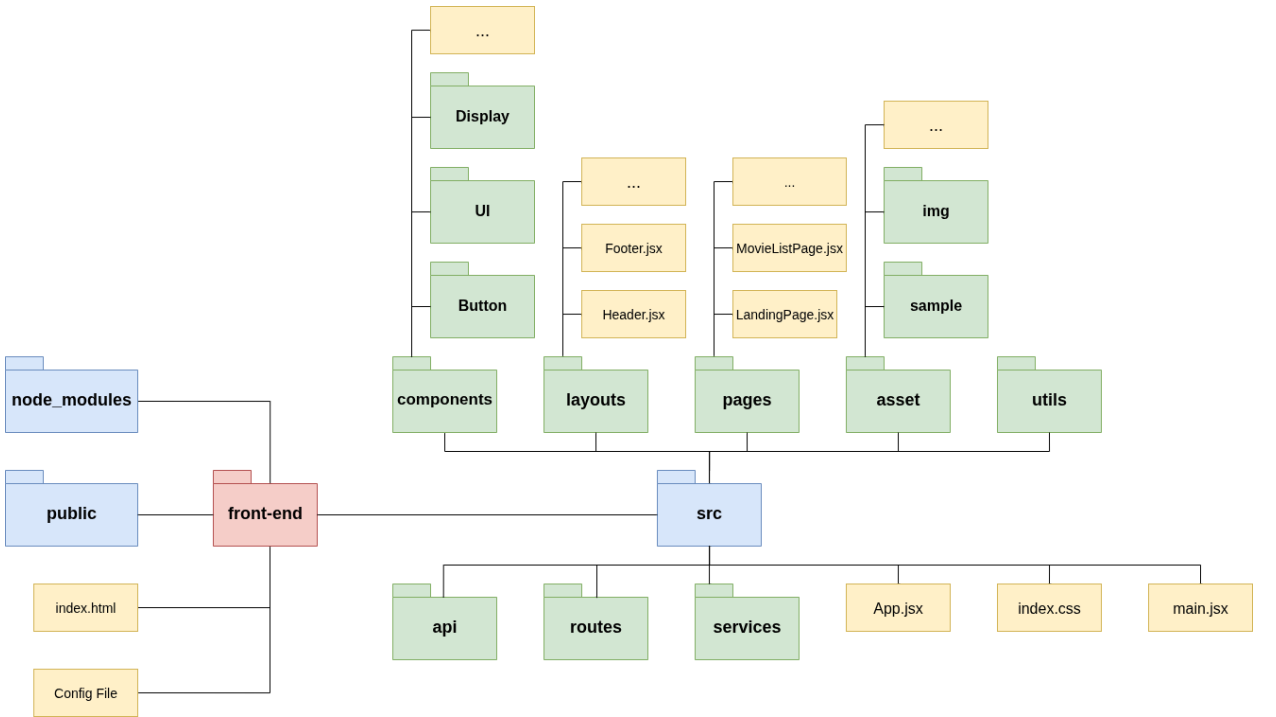
Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

language understanding (NLU) and generation (NLG), allowing users to interact with the system using free-form queries.

- **Deployment:**
 - + The LLM is not hosted on Lumiere's infrastructure but accessed as an external cloud-based service via HTTP-based API calls. The Chat-bot Controller, deployed as part of the server-side application, acts as the intermediary between user queries and the LLM service. Upon receiving user input, the Chat-bot Controller invokes the LLM API, parses its response, and routes the intent to appropriate internal controllers such as the Movie Controller or Branch Controller.
 - + A lightweight internal wrapper is implemented within the Chat-bot Controller to handle LLM API communication, abstracting authentication and formatting logic. This modular integration allows the LLM service to be replaced or extended in the future without affecting core business logic.
- **Specifications:**
 - + Provider: External LLM API service (e.g., Google Gemini API).
 - + Protocol: HTTP-based RESTful API with JSON payloads.
 - + Authentication: API keys securely stored in environment variables; not exposed to the client-side.
 - + Security: The system is designed with an abstraction layer to limit direct access and prevent sensitive data leakage. Sensitive information is not transmitted through the LLM interface.
 - + Performance: Average LLM response time is under 1 second. The system maintains overall interaction latency under 3 seconds for 95% of chatbot requests.
 - + Fallback Mechanism: If the LLM API fails (e.g., timeout, service unavailable), the chatbot responds with a default fallback message and logs the failure for analysis.
 - + Scalability: As the LLM is an external service, it benefits from the provider’s scalability. The internal wrapper includes logic for retry and throttling to handle request surges.

6. Implementation View

6.1 Front-end



node_modules: Contains all third-party libraries and dependencies installed via npm/yarn. Automatically generated – should not be modified manually.

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

public: Contains static files like icons. Files here are publicly accessible and served as-is by the web server.

Config File: Stores project-wide configuration (e.g., environment variables, constants). Used to manage settings or switches that control app behavior.

src: Main source code folder for the frontend application. Includes all logic, layouts, components, and utilities of the project.

api: Stores definitions of API endpoints or base URLs.

Centralizes the configuration for making API requests.

asset: Holds shared static assets like images, and other media.

components: Contains reusable UI components such as buttons, cards, modals,.. Subdivided into smaller categories.

layouts: Stores layout-level components like headers, footers, and main structures of the page.

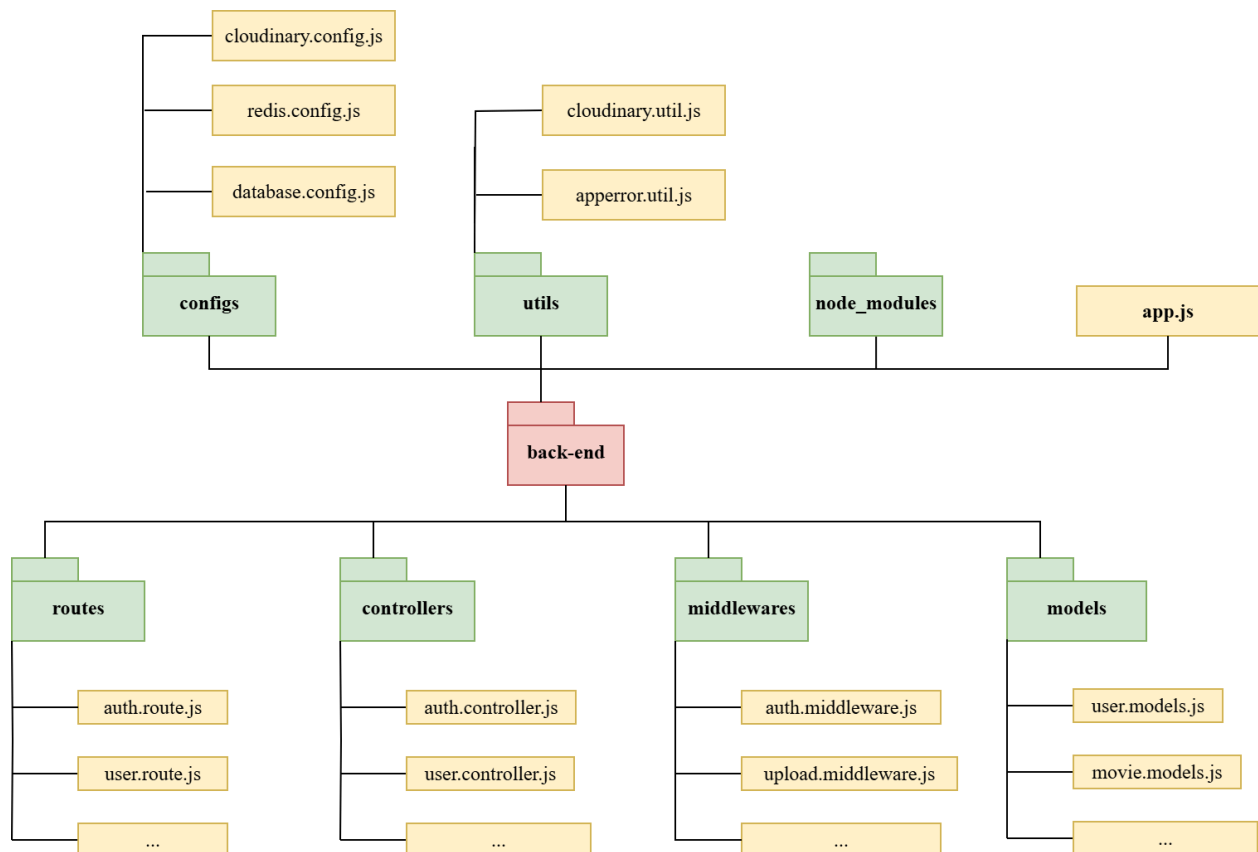
pages: Each file here represents a full page/view in the app. These are mapped to routes.

routes: Defines and manages routing logic for different pages.

services: Handles the logic for calling APIs.

utils: Contains utility/helper functions used globally.

6.2 Back-end



configs: This folder encapsulates configuration files, such as the logic for connecting to the MongoDB database (database.config.js) and setting up external services like Cloudinary, Redis.

node_modules: Contains all third-party libraries and dependencies installed via npm/yarn. Automatically generated – should not be modified manually.

controllers: Contains the application's core business logic. These files receive and process HTTP requests, interact directly with the models to perform data operations, and formulate the final response to send back to the client.

middlewares: This is used for cross-cutting concerns like validating user authentication tokens (auth.middleware.js), checking user roles for authorization, and processing file uploads(upload.middleware.js).

Lumiere Cinema	Version: 1.5
Software Architecture Document	Date: 28/06/2025
lumiere-cinema-sad-v1.5	

models: This directory contains all 13 of the Mongoose schemas that define the application's data structure, validation rules, and relationships between different data entities.

routes: Defines all the application's API endpoints. Each file in this directory maps specific URL paths and HTTP methods to the corresponding controller functions that will handle them.

utils: A collection of reusable helper functions and utility classes, to keep the codebase DRY.

app.js: The main entry point for the back-end application. This file is responsible for initializing the Express server, connecting to the database, applying global middleware, mounting the main router, and starting the server to listen for incoming requests.